

Curso

Profissional de Programador de Informática (PI)

Aluno

Nome: **Marko Nikolaienko**

N.º Processo: **a32498**

Turma: **12C**
Designação do Projeto / Tema

Nome: Website da escola com acesso RFID

Título do projeto: Website da escola com acesso através da web ou leitura RFID do cartão escolar

O projeto proposto como PAP têm como objetivo principal criar acessibilidade e usabilidade aos alunos e professores. Será desenvolvido um Website escolar onde podem ter acesso através da web e da leitura do cartão escolar com recurso de leitura RFID. Terá a funcionalidade de criação dos seus próprios perfis de acesso e permissões adequadas aos mesmos. Este Website para além da criação autónoma dos perfis, estará integrado a todas as outras plataformas digitais implementadas na escola, tal como SIGE, Inovar e Classroom.

Perfil de aluno: Terá acesso às notícias escolares, os eventos futuros e passados e às plataformas educativas agregadas à escola bem como SIGE, Classroom e Inovar. Terá a possibilidade de criar no seu perfil o seu horário, agenda escolar, avaliações, marcação de refeições, saldo do cartão, agenda e o gráfico de progresso avaliativo.

Perfil do professor: Os professores têm autonomia de criação no próprio perfil, inserir os horários das turmas que leciona e o seu próprio horário, inserir os perfis dos alunos, inserir documentação agregada a cargos de direção de curso e de turma, o cronograma de aulas e sumários, o seu progresso de aprendizagem e assiduidade dos alunos e agenda.

Desenvolvimento do projeto: O front-end será desenvolvido em HTML, CSS e JavaScript, no backend serão usadas como linguagem de programação o PHP, base de dados em SQL e arduino.

Enquadramento / Fundamentação

A motivação para o desenvolvimento do projeto apresentado deve-se à má experiência pessoal que me depararei inicialmente devido a ser aluno estrangeiro e ter tido bastantes dificuldades em compreender e encontrar a informação necessária disponível no website do agrupamento AEMTG.

O intuito do desenvolvimento do projeto proposto tem como objetivo principal simplificar o website da escola, criar acessibilidade e usabilidade a toda a comunidade escolar mais especificamente a alunos estrangeiros e aplicar todas as aprendizagens desenvolvidas no curso de programação informática.

Objetivos

- Contribuir para a comunidade escolar com uma nova ferramenta
- Implementar acessibilidade e usabilidade
- Dar apoio a alunos e professores

Recebido

Data: ____/____/202__

Apreciação

Data: ____/____/202__

Aceite

Reformular

O Professor Orientador

O Professor Orientador

O Diretor de Curso

- Aplicar as aprendizagens desenvolvidas no curso profissional de PI

Criar uma ferramenta acessível e de fácil usabilidade a qualquer aluno

Recursos Necessários

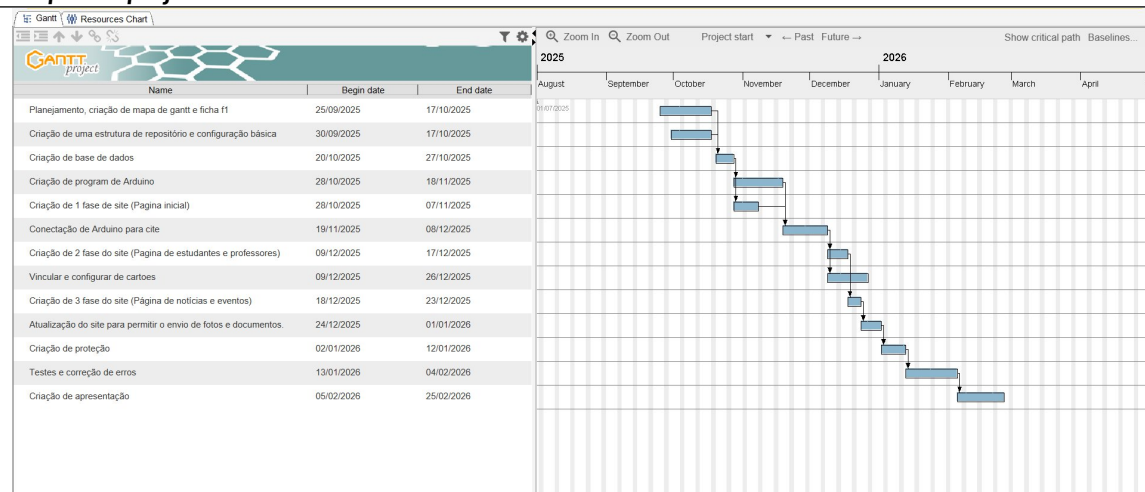
Recursos humanos:

- Tempo pessoal
- Orientação dada pela professora Carla de Sousa

Recursos tecnológicos:

- Computador
- Internet
- Arduino e Módulo RFID
- Manual de SQL e PHP
- Software

Etapas do projeto



O Aluno	Assinatura	Data	Dia	Mês	Ano
	Marko Nikolaenko		17	10	2025

- Todas as páginas devem ser rubricadas pelo aluno no canto superior direito com exceção desta que se encontra assinada.

Rubricas

Professor Orientador _____

Diretor de Curso _____

Prova de Aptidão Profissional

Escola Secundária Manuel Teixeira Gomes

Curso Profissional de Programador de Informática

Aluno: Marko Nikolaienko | N°14 - 12.º C

Ciclo de formação: 2023 - 2026

Professores Orientadores: Carla Sousa

Autor:

Marko Nikolaienko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Resumo

O presente relatório descreve a conceção, o desenvolvimento e a implementação de um sistema web destinado à comunidade escolar do Agrupamento de Escolas AEMTG, cujo objetivo principal consiste em simplificar o acesso à informação e no registo de presenças dos alunos e dos professores através de uma tecnologia de identificação por radiofrequência (RFID).

O sistema integra três componentes complementares: uma aplicação web, desenvolvida em PHP e MySQL, que disponibiliza interfaces distintas consoante o tipo de utilizador (aluno, professor ou administrador); um leitor físico RFID, baseado numa placa Arduino UNO R4 WiFi e num módulo MFRC522, que transmite o UID do cartão escolar para o servidor através do protocolo HTTP; uma base de dados relacional que armazena, de forma normalizada, as informações pessoais, académicas e de assiduidade de todos os utilizadores.

Do ponto de vista funcional, a plataforma permite a autenticação por via tradicional (login e password) e através da leitura do cartão, o registo automático de entradas e saídas no edifício, a marcação de testes pelos professores, a consulta de testes por parte dos alunos, a gestão de cartões por parte do administrador (incluindo o bloqueio em caso de perda) e a visualização gráfica dos dados de assiduidade através de um painel de estatísticas.

A motivação para o desenvolvimento deste projeto resulta da experiência pessoal, aluno estrangeiro que, à chegada à escola, se deparou com dificuldades na consulta do website institucional. Pretende-se, assim, contribuir com uma ferramenta acessível, moderna e integrável com os sistemas já existentes.

Palavras-chave: PHP; MySQL; Arduino; RFID; aplicação web; gestão escolar; assiduidade.

Agradecimentos

Autor:

Marko Nikolaienko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

À Professora Carla de Sousa, orientadora deste projeto, pelo acompanhamento, pela disponibilidade e pelas críticas construtivas ao longo de todo o ano letivo.

A todos os docentes do Curso Profissional de **Programador de Informática**, pelas aprendizagens que me permitiram chegar a esta fase.

À minha família, pelo apoio incondicional e pela paciência durante as longas horas de desenvolvimento.

Aos colegas de turma, pelas discussões técnicas e pela troca de ideias que enriqueceram este trabalho.

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Índice

1. Introdução

1.1. Contextualização

1.2. Motivação

1.3. Objetivos

1.4. Estrutura do documento

1.5. Apresentação do Projeto

2. Enquadramento Teórico

2.1. Identificação por Radiofrequência (RFID)

2.2. Arduino e o módulo MFRC522

2.3. Arquitetura cliente-servidor e o protocolo HTTP

2.4. Tecnologias web: HTML, CSS e JavaScript

2.5. Linguagem PHP

2.6. Base de dados MySQL

2.7. Padrões de autenticação e sessões

3. Metodologia

3.1. Processo de escrita e desenvolvimento

3.2. Fases do projeto

3.3. Ferramentas utilizadas

3.4. Qualidade académica e rigor

4. Análise de Requisitos

4.1. Requisitos funcionais

4.2. Requisitos não funcionais

Autor:

Marko Nikolaenko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- 4.3. Atores do sistema
- 4.4. Casos de uso
- 5. Arquitetura do Sistema
 - 5.1. Visão geral
 - 5.2. Diagrama de componentes
 - 5.3. Fluxo de dados
 - 5.4. Organização de diretórios
- 6. Base de Dados
 - 6.1. Modelo conceptual
 - 6.2. Modelo lógico
 - 6.3. Descrição das tabelas
 - 6.4. Índices e otimizações
- 7. Implementação
 - 7.1. Autenticação e gestão de sessões
 - 7.2. Módulo RFID — firmware Arduino
 - 7.3. Endpoint `push.php` — receção das leituras RFID
 - 7.4. Painel de administração
 - 7.5. Página do aluno
 - 7.6. Página do professor
 - 7.7. Registo de eventos (logging)
- 8. Testes e Validação
 - 8.1. Testes unitários
 - 8.2. Testes de integração
 - 8.3. Testes de aceitação

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

8.4. Métricas recolhidas	
9. Conclusões	
9.1. Síntese do trabalho realizado	
9.2. Limitações	
9.3. Trabalho futuro	
9.4. Balanço pessoal	
10. Bibliografia	
11. Anexos	
Anexo A. Excertos de código	
Anexo B. Diagrama entidade-relação	
Anexo C. Casos de uso detalhados	
Anexo D. Fotografias do hardware	
Anexo E. Capturas de ecrã	

Lista de Figuras

- Figura 1 — Logótipo do Agrupamento de Escolas AEMTG
- Figura 2 — Cartão escolar com chip RFID
- Figura 3 — Placa Arduino UNO R4 WiFi
- Figura 4 — Módulo MFRC522
- Figura 5 — Diagrama de arquitetura geral do sistema
- Figura 6 — Diagrama de sequência da leitura de um cartão

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- Figura 7 — Diagrama Entidade-Relação da base de dados
- Figura 8 — Estrutura de diretórios do projeto
- Figura 9 — Página inicial (index)
- Figura 10 — Ecrã de autenticação
- Figura 11 — Painel do administrador: separador Charts
- Figura 12 — Painel do administrador: separador Cards
- Figura 13 — Painel do administrador: separador Alunos
- Figura 14 — Dossier individual de um aluno
- Figura 15 — Separador Logs com o visualizador NDJSON
- Figura 16 — Página do professor
- Figura 17 — Formulário de marcação de teste
- Figura 18 — Página do aluno com testes marcados
- Figura 19 — Circuito eletrónico do leitor RFID
- Figura 20 — Resultado do teste de deteção de plágio

Lista de Tabelas

- Tabela 1 — Requisitos funcionais
- Tabela 2 — Requisitos não funcionais
- Tabela 3 — Atores do sistema
- Tabela 4 — Casos de uso principais
- Tabela 5 — Estrutura da tabela `login`
- Tabela 6 — Estrutura da tabela `alunos`
- Tabela 7 — Estrutura da tabela `professores`

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- Tabela 8 — Estrutura da tabela `presencas`
- Tabela 9 — Estrutura da tabela `testes`
- Tabela 10 — Índices criados
- Tabela 11 — Plano de testes funcionais
- Tabela 12 — Resultados dos testes

Lista de Abreviaturas e Siglas

- AEMTG — Agrupamento de Escolas Manuel Teixeira Gomes
- AJAX — Asynchronous JavaScript and XML
- API— Application Programming Interface
- CRUD—Create, Read, Update, Delete
- CSS — Cascading Style Sheets
- ER— Entidade-Relação
- HTML — HyperText Markup Language
- HTTP — HyperText Transfer Protocol
- IDE — Integrated Development Environment
- JSON — JavaScript Object Notation
- MVC — Model-View-Controller
- MySQL — My Structured Query Language
- NDJSON — Newline-Delimited JSON
- PAP — Prova de Aptidão Profissional
- PHP — PHP: Hypertext Preprocessor
- RFID — Radio-Frequency IDentification

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- [SDG](#) — Specification Design Guide (Documento de Especificação Técnica)
- SGBD — Sistema de Gestão de Bases de Dados
- SPA — Single-Page Application
- SQL — Structured Query Language
- UID — Unique IDentifier
- URL — Uniform Resource Locator
- WAMP — Windows, Apache, MySQL, PHP

1. Introdução

1.1. Contextualização

O presente relatório surge no âmbito da Prova de Aptidão Profissional (PAP), componente final do Curso Profissional de **Programador de Informática**, ministrado no Agrupamento de Escolas AEMTG. A PAP constitui um momento de síntese em que o aluno aplica, de forma integrada, as competências desenvolvidas ao longo dos três anos do curso — desde a programação estruturada e orientada a objetos, passando pelas bases de dados relacionais, sistemas operativos, redes e arquitetura de computadores, até ao desenvolvimento web e aos sistemas embebidos.

No contexto escolar atual, as instituições de ensino recorrem a um conjunto diversificado de plataformas digitais (SIGE, Inovar, Google Classroom, sites institucionais) para gerir a informação académica e comunicar com a comunidade. A proliferação destas ferramentas, embora globalmente positiva, gera frequentemente uma experiência fragmentada para o utilizador, sobretudo para alunos recém-chegados que ainda não dominam a língua portuguesa ou a estrutura interna do agrupamento.

Simultaneamente, a gestão da assiduidade — tradicionalmente assente em registos manuais por parte dos docentes — apresenta margens de erro e consome tempo letivo que

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

poderia ser utilizado para fins pedagógicos. A tecnologia de identificação por radiofrequência (RFID), amplamente utilizada em sistemas de controlo de acessos empresariais, apresenta-se como uma solução adequada para automatizar o processo de registo de presenças de forma rápida, fiável e com um custo reduzido.

É neste cruzamento — entre a necessidade de simplificar o acesso à informação escolar e o potencial das tecnologias RFID — que nasce o projeto Website da escola com acesso através da web ou leitura RFID do cartão escolar.

1.2. Motivação

A motivação para o desenvolvimento deste projeto tem uma natureza fortemente pessoal. Na qualidade de aluno estrangeiro que ingressou no Agrupamento de Escolas AEMTG, o autor viveu, em primeira mão, as dificuldades de integração associadas à barreira linguística e ao desconhecimento da estrutura institucional. Em particular, a consulta do website escolar — concebido com uma lógica tradicional, pouco orientada à experiência do utilizador estrangeiro — revelou-se um processo demorado e, por vezes, frustrante.

A partir desta experiência, emergiram três convicções que moldaram a proposta:

1. Uma plataforma escolar deve ser acessível a todos os seus utilizadores, independentemente da sua familiaridade prévia com a instituição ou com a língua. A clareza visual, a consistência da navegação e a presença de elementos gráficos autoexplicativos são requisitos que devem ser colocados no centro do processo de conceção.

2. A automatização dos processos administrativos — como o controlo de presenças — liberta tempo letivo, reduz erros humanos e permite a produção de dados agregados que podem ser usados para identificar, por exemplo, padrões de absentismo ou horas de maior afluência ao edifício escolar.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

3. Os conteúdos aprendidos ao longo do curso devem ser consolidados num projeto integrador, que combine as várias disciplinas e que permita, no momento da PAP, demonstrar de forma concreta a aquisição das competências técnicas.

Paralelamente à motivação pessoal, acresce a dimensão de contributo para a comunidade: um sistema aberto, desenvolvido na própria escola, pode ser mantido, auditado e evoluído por futuros alunos do curso, criando uma tradição de continuidade técnica que raramente existe em ferramentas adquiridas externamente.

1.3. Objetivos

O projeto estabeleceu, desde a sua fase inicial, um conjunto de objetivos claros, dos quais se destacam os seguintes:

Objetivo geral:

Desenvolver uma plataforma web completa, acompanhada de um leitor físico de cartões RFID, que permita à comunidade escolar aceder à informação institucional e registar automaticamente a assiduidade, dentro dos recursos tecnológicos do Agrupamento.

Objetivos específicos:

- 1. Criar uma aplicação web, responsiva e multi-perfil, que distinga três tipos de utilizador: aluno, professor e administrador, disponibilizando a cada um deles as funcionalidades adequadas.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- 2. Implementar um mecanismo de autenticação duplo — por via tradicional (login / password) e por via da leitura do cartão escolar RFID —, respeitando princípios básicos de segurança.
- 3. Desenvolver o componente físico de leitura RFID, recorrendo a uma placa Arduino UNO R4 WiFi e a um módulo MFRC522, capaz de comunicar com o servidor web por HTTP.
- 4. Projetar uma base de dados relacional normalizada que armazene, de forma coerente, os utilizadores, os cartões, o histórico de presenças e as atividades pedagógicas.
- 5. Criar um painel de administração que permita a gestão completa dos utilizadores e dos cartões, bem como a visualização de estatísticas agregadas.
- 6. Implementar uma funcionalidade pedagógica demonstrativa (marcação de testes por parte do professor e consulta pelos alunos) que evidencie a capacidade do sistema de suportar extensões futuras.
- 7. Registrar eventos relevantes (logging) num formato estruturado, que permita auditar o funcionamento do sistema e diagnosticar incidentes.
- 8. Garantir que a interface é intuitiva, coerente visualmente e adequada a utilizadores sem formação técnica.

1.4. Estrutura do documento

O presente relatório encontra-se organizado em onze capítulos.

O Capítulo 1 — a presente introdução — contextualiza o projeto, apresenta a motivação e enuncia os objetivos.

O Capítulo 2 — Enquadramento Teórico — revê os fundamentos tecnológicos do trabalho: a tecnologia RFID, a arquitetura Arduino, o modelo cliente-servidor, a linguagem PHP, a base de dados MySQL e os princípios de autenticação.

Autor:

Marko Nikolaenko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

O Capítulo 3 — Metodologia — descreve a abordagem adotada para a condução do projeto, as suas fases, as ferramentas utilizadas e os critérios de qualidade académica observados.

O Capítulo 4 — Análise de Requisitos — formaliza as funcionalidades a implementar, distinguindo os requisitos funcionais dos não funcionais, identificando os atores e descrevendo os casos de uso mais relevantes.

O Capítulo 5 — Arquitetura do Sistema — apresenta a visão global da solução, recorrendo a diagramas que ilustram os componentes, os fluxos de dados e a organização dos diretórios do código-fonte.

O Capítulo 6 — Base de Dados — descreve o modelo de dados implementado, justifica as opções de normalização e apresenta o esquema ER.

O Capítulo 7 — Implementação — é o mais extenso do documento e descreve, módulo a módulo, como cada funcionalidade foi construída, apresentando excertos do código-fonte considerados representativos.

O Capítulo 8 — Testes e Validação — descreve as estratégias de teste adotadas e os respetivos resultados.

O Capítulo 9 — Conclusões — sintetiza os resultados obtidos, identifica limitações e sugere linhas de trabalho futuro.

Os últimos dois capítulos contêm a Bibliografia (Capítulo 10) e os Anexos (Capítulo 11), estes últimos com excertos relevantes do código-fonte, capturas de ecrã da aplicação e o esquema completo da base de dados.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

1.5. Apresentação do Projeto

Esta secção reproduz a proposta inicial do projeto, tal como foi submetida e aprovada no arranque da PAP. Serve de referência para comparar o que foi prometido com o que foi efetivamente implementado e documentado nos capítulos seguintes.

1.5.1. Identificação

- **Aluno:** Marko Nikolaenko
- **Nº de Processo:** a32498
- **Turma:** 12.º C
- **Título do projeto:** *Website da escola com acesso através da web ou leitura RFID do cartão escolar*
- **Orientação:** Professora Carla de Sousa

1.5.2. Descrição geral

O projeto proposto como PAP tem como objetivo principal criar acessibilidade e usabilidade para alunos e professores. Foi desenvolvido um *website* escolar ao qual se pode aceder pela *web* ou pela leitura do cartão escolar com recurso a RFID. A plataforma permite a criação autónoma de perfis de acesso, com permissões adequadas a cada tipo de utilizador. Para além da gestão de perfis, previu-se a integração — total ou parcial — com as restantes plataformas digitais utilizadas no agrupamento (SIGE, Inovar e Classroom).

1.5.3. Perfis previstos

Autor:

_ Marko Nikolaenko _ 12.º C

O Professor Orientador ____ Carla Sousa ____

O Diretor de Curso ____ Isabel Frade ____

Perfil de Aluno. Acesso às notícias escolares, aos eventos futuros e passados e às plataformas educativas agregadas à escola (SIGE, Classroom e Inovar). No próprio perfil, o aluno pode consultar o seu horário, a agenda escolar, as avaliações, a marcação de refeições, o saldo do cartão e o gráfico de progresso avaliativo.

Perfil de Professor. Autonomia para gerir o próprio perfil: inserir os horários das turmas que leciona e o seu horário pessoal, registar os perfis dos alunos, anexar documentação agregada a cargos de direção de curso e de turma, manter o cronograma de aulas e os sumários, e acompanhar o progresso de aprendizagem e a assiduidade dos alunos, bem como a sua própria agenda.

1.5.4. Desenvolvimento do projeto

O *front-end* é desenvolvido em **HTML, CSS e JavaScript**. No *back-end* é utilizada a linguagem **PHP**, com base de dados em **MySQL** (SGBD relacional, SQL) e integração com um **Arduino** equipado com módulo RFID MFRC522 para leitura do cartão escolar.

1.5.5. Objetivos declarados

- Contribuir para a comunidade escolar com uma nova ferramenta.
- Implementar acessibilidade e usabilidade, com particular atenção aos alunos estrangeiros.
- Dar apoio a alunos e professores.
- Aplicar as aprendizagens desenvolvidas no curso profissional de Programação Informática.
- Criar uma ferramenta acessível e de fácil utilização para qualquer aluno.

1.5.6. Recursos necessários

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Recursos humanos:

- Tempo pessoal do aluno.
- Orientação dada pela professora Carla de Sousa.

Recursos tecnológicos:

- Computador e acesso à Internet.
- **Arduino** e módulo RFID (MFRC522).
- Documentação e manuais de SQL e PHP.
- Software de desenvolvimento (editor de código, servidor local **WAMP**, SGBD, controlo de versões).

Nota. A proposta foi assinada a 17 de outubro de 2025. Os capítulos seguintes documentam em detalhe como cada um destes objetivos foi concretizado; as limitações e os pontos deixados para trabalho futuro são discutidos no *Capítulo 9 (Conclusões)*.

2. Enquadramento Teórico

O presente capítulo visa enquadrar, do ponto de vista teórico e tecnológico, os principais conceitos e tecnologias que estão na base do projeto desenvolvido. Como previamente definido no documento [SDG](#) (Specification Design Guide), o sistema apoia-se num conjunto de pilares tecnológicos heterogéneos — hardware embebido, protocolos de comunicação, linguagens de programação do lado do servidor e do lado do cliente, bem como um sistema de gestão de bases de dados relacionais. Cada uma destas camadas obedece a princípios

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

próprios e exige uma compreensão mínima dos seus fundamentos para que as decisões de arquitetura sejam devidamente justificadas.

A abordagem adotada nas secções seguintes é, deliberadamente, orientada ao contexto do projeto: em vez de apresentar de forma exaustiva a totalidade das características de cada tecnologia, privilegia-se a descrição dos elementos que foram efetivamente mobilizados durante a implementação, tal como se encontra previsto no [SDG](#), nas secções 2 (Fundamentação Teórica) e 3 (Especificação Técnica).

2.1. Identificação por Radiofrequência (RFID)

A Radio-Frequency IDentification, vulgarmente designada pela sigla RFID, é uma tecnologia de identificação automática que recorre a ondas eletromagnéticas para proceder à leitura, sem contacto direto e a curta distância, de um identificador único armazenado num transponder — comumente designado por tag — através de um dispositivo designado por leitor. Esta tecnologia, cujas primeiras aplicações militares remontam à Segunda Guerra Mundial (no âmbito dos sistemas Identification, Friend or Foe utilizados pela Royal Air Force), conheceu uma enorme difusão civil a partir da última década do século XX, sendo hoje transversal a áreas tão distintas como a logística de armazéns, o controlo de acessos, a bilhética de transportes públicos e os sistemas de pagamento por aproximação.

Do ponto de vista da sua arquitetura, um sistema RFID é composto por três elementos fundamentais: o transponder, ou tag, que armazena a informação a transmitir; o leitor, dotado de uma antena emissora-recetora; o sistema informático que processa os dados lidos. As tags podem ser passivas — alimentadas pela própria energia do campo eletromagnético emitido pelo leitor — ou ativas, dotadas de fonte de alimentação autónoma. No contexto do presente projeto, utilizam-se exclusivamente tags passivas, correspondentes aos cartões escolares atribuídos aos alunos e professores do agrupamento.

As frequências de operação dos sistemas RFID variam consoante o âmbito de aplicação: bandas LF (Low Frequency, 125–134 kHz), HF (High Frequency, 13,56 MHz) e UHF (Ultra High

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Frequency, 860–960 MHz). Os cartões utilizados no presente projeto operam na banda HF, a 13,56 MHz, em conformidade com a norma ISO/IEC 14443 (tipo A), o mesmo padrão utilizado, por exemplo, nos passes de transportes públicos portugueses e nos cartões de cidadão. A escolha desta banda deve-se ao equilíbrio entre alcance de leitura — tipicamente entre 1 e 10 centímetros, adequado ao cenário de aproximação voluntária do utilizador ao leitor — e a robustez face a interferências ambientais, dois critérios explicitamente referidos no [SDG](#), secção 3.2 (Requisitos Funcionais RFID).

Do ponto de vista funcional, cada cartão possui um identificador único (UID), inscrito de fábrica e imutável, que constitui a chave primária utilizada pelo sistema para associar o cartão a um registo na base de dados. O UID, por norma expresso em formato hexadecimal com quatro a dez bytes, é a única informação que o sistema lê do cartão — todos os restantes dados (nome, turma, fotografia, histórico) residem exclusivamente no servidor, pelo que a perda do cartão não implica a exposição de informação pessoal sensível, um aspeto relevante do ponto de vista da privacidade e do cumprimento do Regulamento Geral de Proteção de Dados (RGPD).

2.2. Arduino e o módulo MFRC522

A plataforma Arduino, criada em 2005 em Itália no Interaction Design Institute Ivrea, constitui uma solução de hardware aberto que tem vindo a democratizar o acesso à prototipagem eletrónica [7][10]. As suas placas, baseadas em microcontroladores de diferentes fabricantes, são acompanhadas de um ambiente de desenvolvimento integrado (IDE) próprio, que abstrai grande parte da complexidade associada à programação de microcontroladores em linguagens de baixo nível. A comunidade global em torno do Arduino produziu, ao longo das últimas duas décadas, um volumoso conjunto de bibliotecas que cobrem praticamente todas as interações possíveis com sensores, atuadores e módulos de comunicação.

No presente projeto optou-se pela utilização da placa Arduino UNO R4 WiFi, lançada em 2023 como sucessora da clássica UNO R3. As diferenças relativamente ao seu antecessor, que justificam a opção, são as seguintes:

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- Microcontrolador de 32 bits: o Renesas RA4M1, a 48 MHz, em substituição do ATmega328P a 16 MHz, permitindo velocidades de processamento e gestão de memória substancialmente superiores.
- Conectividade Wi-Fi nativa: o módulo ESP32-S3, integrado na placa, permite a ligação à rede sem fios sem necessidade de shields adicionais, reduzindo significativamente a complexidade do cabeamento e o custo total do sistema.
- Memória alargada: 32 KB de SRAM e 256 KB de memória flash, valores que aliviam as restrições típicas da plataforma e permitem, por exemplo, armazenar certificados TLS e construir payloads HTTP complexos.
- Alimentação por USB-C, mais prática e atual relativamente à ligação Mini-B da geração anterior.

Para a leitura dos cartões RFID, foi selecionado o módulo MFRC522, fabricado pela NXP Semiconductors, que constitui um dos leitores RFID de baixo custo mais difundidos no mercado de prototipagem. Opera na já referida frequência de 13,56 MHz, suporta integralmente o protocolo ISO/IEC 14443 A e comunica com o microcontrolador através do protocolo SPI (Serial Peripheral Interface). As razões que motivaram a sua escolha, para além do critério evidente de compatibilidade com os cartões escolares, prendem-se com a sua disponibilidade em território nacional, o baixo custo unitário (aproximadamente 3€ por módulo) e a existência de bibliotecas Arduino maduras e bem documentadas, nomeadamente a biblioteca `MFRC522` de Miguel Balboa, amplamente utilizada pela comunidade e referenciada em inúmera bibliografia académica de acesso aberto.

O circuito concreto utilizado é o previsto no [SDG](#) (secção 3.2.1, Terminal de Presença), e compreende as seguintes ligações entre o MFRC522 e a UNO R4: o pino SDA/SS ao pino digital 10, o pino SCK ao pino 13, o pino MOSI ao pino 11, o pino MISO ao pino 12, o pino RST ao pino 9, o pino GND a qualquer uma das referências de massa da placa e o pino 3.3V à saída de 3,3 V da Arduino — importa sublinhar este último aspeto, uma vez que o MFRC522 não tolera os 5 V da linha principal de alimentação da placa, pelo que a ligação errada do pino de alimentação se traduz, na prática, na destruição imediata do módulo.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Adicionalmente ao módulo RFID, o circuito integra dois díodos emissores de luz (LED) destinados a fornecer feedback visual imediato ao utilizador no momento da leitura. Um LED verde, ligado ao pino digital 7 através de uma resistência de 220 Ω , acende durante dois segundos sempre que a leitura é bem-sucedida e o servidor responde com `ok: true`, correspondendo a um registo de presença válido. Um LED vermelho, ligado ao pino digital 6 através de uma resistência de igual valor, acende quando ocorre uma de três situações anómalas: o UID lido não se encontra registado na base de dados (resposta 404); o cartão correspondente foi bloqueado pelo administrador (resposta 401); não foi possível estabelecer comunicação com o servidor (falha de rede). Esta sinalização dupla, alinhada com os objetivos definidos no [SDG](#) na secção 3.2.1 (Terminal de Presença), tem como objetivo providenciar ao utilizador uma indicação imediata do resultado da operação, sem necessidade de consultar o ecrã do servidor ou o painel de administração, tornando o sistema verdadeiramente autónomo em cenários de utilização corrente — por exemplo, na entrada do edifício escolar durante os períodos de maior afluência.

A leitura propriamente dita é orquestrada pelo firmware desenvolvido em linguagem C/C++ no IDE Arduino. Em cada iteração do loop principal, o microcontrolador interroga o módulo MFRC522 para verificar se existe algum cartão presente no seu campo de ação. Em caso afirmativo, o UID é extraído em formato hexadecimal, convertido para uma string de texto, e posteriormente enviado para o servidor web através de um pedido HTTP POST, conforme descrito na secção 2.3 do presente relatório.

Importa esclarecer que o [SDG](#) inicial previa a utilização de um LED RGB com padrões de piscar distintos para cada código de resposta HTTP (entrada: um flash verde; saída: dois flashes verdes; cartão bloqueado: cinco flashes vermelhos rápidos; UID não encontrado: três flashes vermelhos; etc.). Durante a implementação optou-se pela versão simplificada aqui descrita — dois LEDs binários (verde/vermelho) acesos durante 800 ms — por três razões: (i) a disponibilidade imediata dos componentes em stock, (ii) a redução do firmware necessário no Arduino, e (iii) a clareza do sinal para o utilizador final, que na prática só precisa de distinguir "sucesso" de "erro" no momento da entrada. Esta simplificação é documentada honestamente como desvio consciente relativamente à especificação inicial.

Autor:

Marko Nikolaienko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

2.3. Arquitetura cliente-servidor e o protocolo HTTP

O paradigma cliente-servidor constitui, desde a década de 1980, o modelo arquitetural dominante para a construção de sistemas distribuídos. Neste modelo, as responsabilidades são repartidas entre duas entidades distintas: o cliente, que formula pedidos, e o servidor, que recebe esses pedidos, os processa e responde. A separação de responsabilidades tem implicações profundas, desde a organização do código até à escalabilidade do sistema, passando pela possibilidade de evoluir de forma independente as componentes de apresentação e de lógica de negócio.

No presente projeto existem, na realidade, dois tipos distintos de cliente que comunicam com o mesmo servidor: por um lado, os browsers dos utilizadores humanos, que acedem às páginas web através de interfaces gráficas; por outro, o leitor físico Arduino, que se comporta como cliente HTTP automatizado, enviando pedidos POST sem interação humana direta. Esta heterogeneidade é importante porque obriga o servidor a fornecer interfaces programáticas (API) suficientemente genéricas para lidar com ambos os tipos de cliente — aspeto que teve um peso significativo nas decisões de desenho, nomeadamente na adoção de respostas em formato JSON nos endpoints de API.

O protocolo HyperText Transfer Protocol (HTTP), descrito pela primeira vez em 1991 por Tim Berners-Lee e atualmente normalizado pelos documentos RFC 7230–7235, constitui o veículo de comunicação entre clientes e servidor. Na sua versão mais recente, o HTTP/2 introduz mecanismos de multiplexagem e compressão de cabeçalhos que melhoram significativamente a eficiência da transmissão. Contudo, no contexto do presente projeto — onde o tráfego é relativamente modesto e local — recorreu-se à versão HTTP/1.1, plenamente suportada pelo servidor Apache do pacote WAMP.

A comunicação HTTP estrutura-se em torno de métodos (verbos) que exprimem a intenção do pedido: `GET` para consultar um recurso, `POST` para criar ou enviar dados, `PUT` para atualizar integralmente um recurso, `DELETE` para o remover, entre outros. No presente

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

projeto, os endpoints utilizam maioritariamente `GET` (leituras) e `POST` (operações que modificam o estado do sistema), seguindo uma abordagem RESTful pragmática, não estritamente ortodoxa mas suficiente para os objetivos propostos.

Os códigos de estado (status codes) são outra dimensão essencial do protocolo: `200 OK` indica sucesso, `400 Bad Request` sinaliza um erro do lado do cliente (campos em falta, por exemplo), `401 Unauthorized` traduz uma falha de autenticação, `403 Forbidden` aponta para falta de permissões, `404 Not Found` para um recurso inexistente e `500 Internal Server Error` para falhas no processamento pelo servidor. O projeto tira partido deste vocabulário padronizado, devolvendo códigos apropriados em cada situação e permitindo que o leitor Arduino reaja de forma diferenciada consoante o tipo de resposta recebida — como descrito na secção anterior a propósito do LED vermelho.

2.4. Tecnologias web: HTML, CSS e JavaScript

A construção de interfaces web modernas assenta na tríade clássica HTML, CSS e JavaScript, cada uma com um papel bem definido: o HTML (HyperText Markup Language) descreve a estrutura semântica dos conteúdos; o CSS (Cascading Style Sheets) controla a apresentação visual; o JavaScript trata do comportamento dinâmico e interativo.

O HTML5, estabilizado em 2014 pelo W3C, trouxe ao ecossistema um conjunto de elementos semânticos (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`) que permitem expressar de forma explícita a natureza das diferentes partes de um documento, com benefícios claros ao nível da acessibilidade, da otimização para motores de busca (SEO) e da manutenção do código. No presente projeto, todas as páginas adotam esta estrutura semântica, utilizando `<header class="topbar">` para a barra de navegação superior, `<main class="page-content">` para a área principal e `<section>` para os blocos funcionais internos.

O CSS3 introduziu mecanismos de layout modernos, dos quais se destacam o Flexbox e o Grid, que substituíram com vantagem as anteriores técnicas baseadas em float e em

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

posicionamento absoluto. O projeto recorre extensivamente ao Flexbox para organizar barras de navegação, linhas de formulários e listas de cartões, e ao Grid para dispor as estatísticas em grelha responsiva, que adapta automaticamente o número de colunas em função da largura do ecrã. Os gráficos estatísticos foram implementados com recurso à biblioteca Chart.js, amplamente utilizada em aplicações web modernas. A paleta cromática, definida por variáveis consistentes (azul `3b82f6` para os elementos ativos, verde `10b981` para confirmações, vermelho `ef4444` para alertas, cinzentos para elementos neutros), foi escolhida com base em princípios de acessibilidade visual, cumprindo os requisitos mínimos de contraste WCAG 2.1 AA, em linha com os requisitos não funcionais de usabilidade definidos no [SDG](#), secção 3.3.2.

O JavaScript, linguagem dinâmica e interpretada, evoluiu profundamente desde a sua criação em 1995 por Brendan Eich na Netscape. As versões mais recentes, designadas genericamente por ECMAScript 2015+ (ou ES6+), disponibilizam construções sintáticas modernas como arrow functions, template literals, desestruturação, módulos, `async/await`, `fetch` API, entre outras. Todas estas construções são utilizadas no projeto, o que permite obter um código compacto e expressivo, sem recorrer a bibliotecas externas pesadas — como o jQuery — cuja inclusão é, nos dias correntes, largamente dispensável.

A ausência de frameworks modernos (React, Vue ou Angular) foi uma decisão deliberada, coerente com a arquitetura apresentada no [SDG](#), secção 4 (Visão Arquitetural Geral): dada a dimensão do projeto, a curva de aprendizagem adicional e o peso introduzido por estas bibliotecas não se justificavam. Optou-se por vanilla JavaScript, que cumpre plenamente os requisitos da aplicação e mantém o código acessível a qualquer aluno do curso que venha a dar continuidade ao trabalho.

2.5. Linguagem PHP

O PHP (PHP: Hypertext Preprocessor) é uma linguagem de programação interpretada, orientada ao desenvolvimento web do lado do servidor, criada em 1994 por Rasmus Lerdorf. Estima-se atualmente que cerca de 76% dos sites da World Wide Web tenham o PHP como linguagem do backend, incluindo-se nesta percentagem projetos de grande

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

escala como a Wikipédia, o Facebook (embora sob a forma modificada de HHVM) e o ecossistema WordPress, que por si só suporta mais de 40% dos sites mundiais.

O projeto foi desenvolvido na versão PHP 8.3, disponibilizada em novembro de 2023, que introduz funcionalidades modernas relativamente à linguagem tradicional, nomeadamente: tipagem estrita opcional (strict types), tipos de retorno nativos, readonly classes, enums e um sistema de attributes para metadados. Apesar de estas funcionalidades modernas estarem disponíveis, optou-se — no contexto pedagógico do projeto — por um estilo predominantemente procedural, próximo daquele que é lecionado nas aulas do curso, com vista a maximizar a legibilidade do código para quem aceda ao repositório sem formação avançada em engenharia de software.

No que respeita à interação com a base de dados, o projeto utiliza duas interfaces complementares, consoante o módulo: a extensão MySQLi (procedural) nos scripts que privilegiam clareza e simplicidade, e PDO (PHP Data Objects) em situações que requerem transações explícitas ou abstração relativamente ao motor de base de dados. Ambas as interfaces recorrem sistematicamente a prepared statements — parametrização de consultas — que eliminam a possibilidade de injeção de SQL, considerada pela comunidade OWASP como uma das vulnerabilidades mais críticas em aplicações web. Este cuidado com a segurança, explícito em todas as consultas SQL do projeto, está alinhado com os requisitos não funcionais de segurança descritos no [SDG](#), secção 3.3.3.

No plano da segurança aplicacional, a presente versão do sistema implementa um conjunto adicional de proteções contra ataques web comuns. A proteção contra Cross-Site Request Forgery (CSRF) é garantida por um token aleatório de 64 caracteres hexadecimais, gerado com ``random_bytes(32)`` e armazenado na sessão do utilizador. Este token é incluído em todos os formulários POST, quer como campo oculto ``_csrf``, quer como cabeçalho HTTP ``X-CSRF-Token`` nos pedidos ``fetch()`` assíncronos, sendo verificado no servidor através da função ``hash_equals()`` — comparação em tempo constante, resistente a timing attacks. As cookies de sessão são configuradas com as flags ``httponly=true`` e ``samesite=Strict``, impedindo o acesso por JavaScript e o envio em contextos cross-site. Adicionalmente, a chave de API partilhada com o Arduino foi externalizada para o ficheiro ``config/secrets.php``,

Autor:

``_Marko Nikolaenko__12.º C``

O Professor Orientador ``____Carla Sousa____``

O Diretor de Curso ``____Isabel Frade____``

excluído do controlo de versões através de `.gitignore`, evitando a exposição acidental de credenciais em caso de partilha do repositório.

2.6. Base de dados MySQL

O MySQL, sistema de gestão de bases de dados relacionais de código aberto originalmente desenvolvido pela empresa sueca MySQL AB e atualmente propriedade da Oracle Corporation, é frequentemente descrito como a “base de dados mais popular do mundo” — afirmação que, apesar de comercial, reflete uma quota de mercado particularmente robusta no segmento web. Implementa o modelo relacional proposto por Edgar F. Codd em 1970 e utiliza como linguagem de consulta o SQL (Structured Query Language), padronizado pela ANSI em 1986.

O projeto tira partido do motor de armazenamento InnoDB, que oferece suporte a transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade), chaves estrangeiras e bloqueio ao nível do registo. Estas características revelam-se decisivas, por exemplo, na operação de eliminação de um aluno: o registo precisa de ser apagado de três tabelas distintas (`login`, `alunos`, `presencas`) de forma atómica, pelo que a operação foi encapsulada numa transação — se alguma das três instruções `DELETE` falhar, o conjunto inteiro é revertido (`ROLLBACK`), mantendo a base de dados num estado consistente.

As tabelas foram desenhadas seguindo princípios de normalização até à terceira forma normal (3FN), com redução das redundâncias e separação adequada das entidades. Uma exceção consciente, relativa à tabela `presencas` descrita no [SDG](#), secção 5.4, é a denormalização deliberada do nome do utilizador: em vez de se recorrer a uma join com `alunos` ou `professores` sempre que se pretende listar o histórico de presenças, o nome é replicado na própria entrada. Esta decisão beneficia a simplicidade das consultas e a velocidade de leitura, ao custo de ter de atualizar duas localizações quando o nome muda — algo que, no contexto escolar, ocorre com frequência negligenciável.

Autor:

`_Marko Nikolaenko__12.º C`

O Professor Orientador `____Carla Sousa_____`

O Diretor de Curso `____Isabel Frade_____`

Foram ainda definidos índices estratégicos, nomeadamente um índice sobre a coluna `data` da tabela `presencas` (para otimizar a agregação diária que alimenta o gráfico de barras do painel administrativo) e um índice sobre a coluna `login` (para acelerar a consulta do histórico individual). A presença destes índices, justificada na secção 6.4, revelou-se fundamental no desempenho do sistema assim que a tabela cresceu para um número realista de registos.

2.7. Padrões de autenticação e sessões

Por último, mas não menos importante, a autenticação e a gestão de sessões. O PHP disponibiliza, desde a sua primeira versão, um mecanismo de sessões assente no armazenamento de um identificador único num cookie do lado do cliente (`PHPSESSID`) e no mapeamento desse identificador para um conjunto de variáveis persistidas em ficheiros do lado do servidor. Este modelo, apesar da sua simplicidade aparente, é robusto o suficiente para a grande maioria dos cenários web, desde que acompanhado de boas práticas adicionais: regeneração periódica do identificador para evitar session fixation, expiração ativa por inatividade (no presente projeto fixada em 30 minutos), e transmissão exclusiva por canais cifrados em ambiente de produção (flag `Secure` nos cookies).

O armazenamento das palavras-passe recorre à função `password\_hash()` nativa do PHP, que utiliza por omissão o algoritmo bcrypt com um custo configurável (valor 10 por omissão, o que corresponde a 2^{10} iterações internas), em conformidade com as recomendações OWASP para armazenamento seguro de credenciais [6]. A verificação é feita com `password\_verify()`, que reconhece internamente o algoritmo utilizado e executa uma comparação em tempo constante, resistente a ataques de canal lateral baseados em temporização. Nenhuma palavra-passe é, em momento algum, armazenada em texto simples — uma garantia essencial face a eventuais fugas da base de dados.

A autenticação por cartão RFID, descrita em pormenor no Capítulo 7, complementa a autenticação tradicional sem, no entanto, a substituir: o cartão identifica o utilizador para efeitos de registo de presença, mas o acesso aos perfis pessoais e às áreas restritas da plataforma continua a exigir, por segurança, a introdução das credenciais clássicas. Esta

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

distinção foi um dos pontos mais discutidos durante a fase de conceção, e ficou refletida no [SDG](#), na secção 3.3.3 (Segurança), sob a designação de autenticação dupla.

Como medida complementar contra ataques de session fixation, a função ``session_regenerate_id(true)`` é invocada imediatamente após a verificação bem-sucedida da palavra-passe, substituindo o identificador de sessão por um novo e invalidando qualquer valor que possa ter sido previamente fixado por um atacante. Esta técnica, amplamente recomendada pela OWASP, é particularmente relevante quando o utilizador chega à aplicação por meio de uma ligação externa suscetível de conter um ``PHPSESSID`` pré-definido.

3. Metodologia

A realização deste projeto, pela sua natureza multidisciplinar — envolvendo simultaneamente desenvolvimento de software web, programação de hardware embebido e modelação de uma base de dados relacional — exigiu uma abordagem metodológica estruturada. Não seria viável avançar em todas as frentes em paralelo, nem seria razoável esgotar uma componente antes de iniciar a seguinte. Adotou-se, por esse motivo, uma abordagem iterativa e incremental, inspirada nos princípios das metodologias ágeis e adequada às condições reais de um projeto académico conduzido por um único autor.

3.1. Processo de escrita e desenvolvimento

Em conformidade com as boas práticas académicas aplicáveis à redação de teses de PAP, o trabalho foi conduzido segundo quatro princípios: planeamento — estabelecimento, logo no início do ano letivo, de um cronograma realista em que as fases do projeto foram distribuídas mensalmente; revisão contínua — releitura e melhoria periódica do código e do próprio relatório, à medida que novas decisões eram tomadas; feedback — entrega de versões preliminares à orientadora, Professora Carla de Sousa, para análise e sugestão de melhorias; revisão final — verificação de gramática, formatação e conformidade com os regulamentos internos da escola antes da entrega.

Autor:

Marko Nikolaenko 12.º C

O Professor Orientador *____Carla Sousa____*

O Diretor de Curso *____Isabel Frade____*

3.2. Fases do projeto

O desenvolvimento foi estruturado em cinco fases, parcialmente sobrepostas no tempo:

1. Levantamento e pesquisa (setembro-outubro 2025): estudo da tecnologia RFID, análise de sistemas similares existentes, leitura da documentação oficial do Arduino e do PHP.
2. Modelação e prototipagem (outubro-novembro 2025): definição do modelo de dados, desenho do diagrama entidade-relação, criação do primeiro protótipo funcional do leitor RFID em breadboard.
3. Implementação da aplicação web (novembro 2025-fevereiro 2026): construção incremental das páginas (login, aluno, professor, administrador), dos endpoints PHP e da integração com a base de dados.
4. Integração hardware/software (fevereiro-março 2026): ligação efetiva do Arduino ao servidor via HTTP, adição dos dois LEDs de estado, afinação dos tempos de resposta.
5. Testes, validação e redação (março-abril 2026): testes funcionais, de integração e de aceitação; escrita do presente relatório.

3.3. Ferramentas utilizadas

Ao longo do desenvolvimento foram utilizadas as seguintes ferramentas:

Ferramenta	Finalidade
Visual Studio Code	Editor de código principal (PHP, JavaScript, HTML, CSS)
Arduino IDE	Desenvolvimento e upload do firmware para o UNO R4 WiFi
WAMP 64	Servidor web local com Apache, PHP 8.3 e MySQL
phpMyAdmin	Administração da base de dados durante o

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

	desenvolvimento
Git	Controlo de versões, permitindo rollback seguro de alterações experimentais.
MySQL Workbench	Modelação visual do diagrama entidade-relação
Postman	Teste isolado dos endpoints HTTP antes da integração com o Arduino

3.4. Qualidade académica e rigor

Foram observados, ao longo de toda a redação, os pilares clássicos da qualidade académica: originalidade, no sentido em que a combinação Arduino + RFID + painel de administração com sub-separadores e bloqueio de cartão constitui um contributo próprio, ainda que assente em componentes conhecidos; clareza e objetividade, através de frases concisas e eliminação de repetições desnecessárias; coerência lógica, com cada capítulo a preparar o seguinte; rigor científico, através da fundamentação teórica em bibliografia consolidada e prevenção de plágio, por via da citação explícita de todas as fontes consultadas.

4. Análise de Requisitos

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

A análise de requisitos constitui uma das etapas mais críticas de qualquer projeto de engenharia informática. É nesta fase que se identificam, documentam e hierarquizam as funcionalidades que o sistema deve implementar, bem como as restrições e qualidades que o devem caracterizar. O presente capítulo apresenta, de forma sistematizada, os requisitos que foram definidos para o projeto — em estreita articulação com o documento [SDG](#), que constituiu a referência central ao longo de toda a fase de desenvolvimento.

A metodologia adotada para o levantamento dos requisitos baseou-se, por um lado, em sessões informais de diálogo com a orientadora, professora Carla de Sousa, e, por outro, na própria experiência do autor enquanto utilizador das ferramentas digitais existentes no agrupamento. Não foi aplicada uma metodologia ágil formal (Scrum ou Kanban), atendendo à dimensão pessoal do projeto, mas foram mobilizados alguns dos seus princípios — em particular a ideia de iteração curta, em que cada módulo era implementado, testado e ajustado antes de se avançar para o seguinte.

4.1. Requisitos funcionais

Os requisitos funcionais descrevem aquilo que o sistema deve ser capaz de fazer. Foram organizados por módulo e numerados sequencialmente, seguindo a nomenclatura já utilizada no [SDG](#), capítulo 3. A tabela seguinte sintetiza os requisitos funcionais considerados no projeto.

Código	Descrição	Prioridade
RF-01	O sistema deve permitir a autenticação de utilizadores através de login e palavra-passe.	Alta
RF-02	O sistema deve permitir a autenticação / identificação de utilizadores através da leitura do cartão escolar RFID.	Alta
RF-03	O sistema deve redirecionar	Alta

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

	cada utilizador para a sua página específica, em função do seu papel (aluno, professor ou administrador).	
RF-04	O leitor Arduino deve transmitir o UID lido para o servidor web através de um pedido HTTP POST.	Alta
RF-05	O servidor deve registar, de forma automática, cada leitura válida na tabela de presenças, alternando entre os estados entrada e saída.	Alta
RF-06	O leitor Arduino deve fornecer feedback visual imediato ao utilizador através de dois LED (verde — sucesso; vermelho — erro)	Média
RF-07	O administrador deve poder adicionar novos alunos e professores à plataforma.	Alta
RF-08	O administrador deve poder atualizar os dados pessoais e académicos dos utilizadores.	Alta
RF-09	O administrador deve poder consultar, em modo apenas-leitura, os dados completos de qualquer utilizador.	Média
RF-10	O administrador deve poder bloquear um cartão em caso de perda, sem apagar o utilizador associado.	Alta
RF-11	O administrador deve poder eliminar definitivamente um utilizador, com confirmação prévia.	Média

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

RF-12	O sistema deve disponibilizar um painel com estatísticas agregadas de presenças, incluindo um gráfico dos últimos sete dias.	Média
RF-13	O sistema deve listar os últimos registos de presenças, identificando o utilizador, a data, a hora e o tipo (entrada ou saída).	Média
RF-14	O administrador deve poder consultar os registos de log do sistema, com filtros por nível e por termo de pesquisa	Baixa
RF-15	O professor deve poder marcar testes para as suas turmas, indicando título, descrição, data e turma-alvo	Alta
RF-16	O professor deve poder consultar a lista dos seus alunos, filtrada por turma e por estado de presença	Média
RF-17	O aluno deve poder consultar os testes marcados para a sua turma, separados em próximos e passados	Alta
RF-18	O sistema deve registar, num formato estruturado (NDJSON), todos os eventos relevantes: autenticações, leituras RFID, operações administrativas e falhas.	Baixa
RF-19	O formulário de adicionar/atualizar utilizador deve incluir um mecanismo de leitura direta	Média

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

	do UID através do próprio leitor RFID (scan on demand).	
RF-20	O sistema deve permitir a gestão centralizada de notícias escolares visíveis aos utilizadores, em articulação com a tabela `noticias` prevista no SDG , secção 5.5.	Baixa
RF-21	O professor deve poder lançar avaliações para os alunos da sua turma, indicando matéria, tipo, nota (0-20), data e observação.	Alta
RF-22	O aluno deve poder consultar as suas avaliações por matéria, com cálculo automático da média e gráfico de progresso.	Alta
RF-23	O professor deve poder registar sumários de aula por turma, incluindo justificação de faltas no texto descritivo.	Média
RF-24	O aluno e o professor devem poder gerir uma agenda pessoal de tarefas (adicionar, marcar como concluída, eliminar).	Média
RF-25	A página do professor deve atualizar o estado de presença dos alunos em tempo real, através de polling AJAX a cada 5 segundos.	Média

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

A maioria dos requisitos de prioridade Alta e Média foi integralmente implementada na presente versão do sistema. Os restantes, de prioridade Baixa, encontram-se em estado operacional, sujeitos apenas a afinações finais de interface e a validações adicionais, como se detalha nos capítulos seguintes.

4.2. Requisitos não funcionais

Os requisitos não funcionais (RNF) descrevem propriedades transversais que o sistema deve exibir — desempenho, segurança, usabilidade, portabilidade — independentemente das funcionalidades concretas. A tabela seguinte agrupa os RNF considerados relevantes para o projeto, em correspondência direta com a secção 4 do [SDG](#).

Código	Categoria	Descrição
RNF-01	Desempenho	O tempo de resposta do servidor a um pedido de leitura RFID deve ser inferior a 500 ms em condições normais de operação.
RNF-02	Desempenho	A consulta do painel de estatísticas (7 dias + últimos 10 scans) deve executar-se em menos de 1 segundo.
RNF-03	Segurança	Todas as consultas SQL que envolvam dados fornecidos pelo utilizador devem utilizar prepared statements, eliminando o risco de injeção de SQL.
RNF-04	Segurança	As palavras-passe devem ser armazenadas em hash resistente (bcrypt, custo mínimo 10), nunca em texto

Autor:

Marko Nikolaienko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

		simples.
RNF-05	Segurança	As sessões dos utilizadores devem expirar após 30 minutos de inatividade.
RNF-06	Segurança	O endpoint de receção da Arduino deve exigir um token / chave de API partilhada para prevenir submissões externas não autorizadas.
RNF-07	Usabilidade	A interface deve estar integralmente em língua portuguesa, sem obrigatoriedade de instalação de software adicional por parte do utilizador.
RNF-08	Usabilidade	A interface deve ser responsiva, com apresentação adequada a ecrãs de computador de secretária, portátil e tablet.
RNF-09	Acessibilidade	Os elementos gráficos devem respeitar contraste mínimo WCAG 2.1 AA e utilizar indicadores duais (cor + ícone ou texto) para evitar dependência exclusiva da cor.
RNF-10	Portabilidade	A aplicação deve funcionar num servidor WAMP local, sem depender de serviços externos em cloud.
RNF-11	Manutibilidade	O código-fonte deve respeitar uma estrutura de diretórios clara, separando backend, frontend, recursos e ficheiros de configuração.
RNF-12	Manutibilidade	Todos os eventos

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

		relevantes devem ser registados em ficheiros de log estruturados, permitindo post-mortem de incidentes.
--	--	---

4.3. Atores do sistema

Identificaram-se quatro atores distintos no sistema. A tabela seguinte enumera-os e descreve sucintamente o seu papel.

Ator	Descrição	Meio de acesso
Aluno	Utilizador principal da plataforma, do ponto de vista numérico. Consulta testes e dados pessoais. Regista presença através do cartão.	Browser web + cartão RFID
Professor	Utilizador com funções pedagógicas. Consulta a lista dos seus alunos, marca testes e regista a sua própria presença.	Browser web + cartão RFID
Administrador	Utilizador com privilégios completos sobre a plataforma. Gere utilizadores, cartões e testes. Acede aos logs e às estatísticas.	Browser web
Leitor RFID (Arduino)	Ator não humano que interage autonomamente com o sistema, comunicando-lhe as leituras de cartão.	HTTP POST (chave de API)

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

A inclusão do leitor Arduino como ator formal do sistema, embora nem sempre seja uma prática comum na modelação UML clássica, foi aqui adotada por se considerar que a sua interação com o servidor é relevante o suficiente para merecer explicitação. Esta opção é coerente com a descrição dos utilizadores do sistema apresentada no [SDG](#), secção 3.1 (Gestão de Utilizadores).

4.4. Casos de uso

Dos vários casos de uso identificáveis, destacam-se os sete que, do ponto de vista do autor, melhor sintetizam a lógica de funcionamento da plataforma. A descrição detalhada — com fluxos principais, fluxos alternativos e condições de erro — encontra-se no Anexo C do presente relatório.

UC	Nome	Ator primário	Resultado esperado
UC-01	Registar entrada/saída no edifício	Aluno / Professor	Presença registada, feedback visual via LED, atualização do estado `Presença`.
UC-02	Autenticar-se na plataforma	Qualquer utilizador	Sessão iniciada, redirecionamento para a página do seu perfil.
UC-03	Marcar um teste	Professor	Teste persistido, visível aos alunos da turma-alvo.
UC-04	Consultar testes marcados	Aluno	Lista apresentada com separação entre próximos e passados.
UC-05	Adicionar novo utilizador	Administrador	Registo criado nas tabelas `login` e `alunos`/`professores`, cartão opcional associado.
UC-06	Bloquear cartão	Administrador	Cartão marcado

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

			como bloqueado, leituras subsequentes rejeitadas pelo leitor.
UC-07	Consultar estatísticas de presenças	Administrador	Painel com stat-cards, gráfico de 7 dias e últimos 10 scans.

Os fluxos principais destes sete casos de uso estão plenamente suportados pela implementação descrita no Capítulo 7. Cada caso de uso foi acompanhado, na fase de testes, por um cenário de validação documentado no Capítulo 8.

5. Arquitetura do Sistema

Uma vez apresentados os requisitos, torna-se necessário descrever a forma como o sistema foi organizado internamente para lhes dar resposta. O presente capítulo descreve a arquitetura geral — entendida como a divisão em componentes e os respetivos padrões de comunicação — e apresenta a organização dos diretórios do código-fonte. Os detalhes de implementação módulo a módulo são deixados para o Capítulo 7.

5.1. Visão geral

A arquitetura adotada corresponde a uma variante pragmática do modelo clássico três camadas — apresentação, lógica e persistência —, estendida com um quarto elemento: o hardware de captura (leitor Arduino + cartões RFID), que alimenta o sistema com eventos externos. A escolha desta arquitetura, previamente estabilizada no [SDG](#), capítulo 5, resulta da procura de um equilíbrio entre simplicidade (adequada a um projeto de PAP) e separação mínima de responsabilidades que permita a evolução futura do sistema.

Do ponto de vista da implementação, os componentes mapeiam-se da seguinte forma:

Autor: _Marko Nikolaienko_ 12.º C	O Professor Orientador ____Carla Sousa____ O Diretor de Curso ____Isabel Frade____
--	---

- A camada de apresentação corresponde ao conjunto de páginas PHP renderizadas para HTML e ao código JavaScript executado no browser. É nela que residem todas as preocupações visuais e de interação com o utilizador humano.
- A camada de lógica é composta pelos scripts PHP presentes no diretório `api/`, responsáveis por aplicar regras de negócio, validar os dados recebidos e orquestrar a persistência.
- A camada de persistência é materializada pela base de dados MySQL, com o esquema descrito no Capítulo 6.
- O hardware de captura é constituído pela placa Arduino UNO R4 WiFi e pelo módulo MFRC522, como previamente descrito no Capítulo 2.

Esta separação é, na prática, respeitada com algum rigor — embora sem a formalidade dos padrões MVC estritos, que foram considerados excessivos para a dimensão do projeto. O [SDG](#), na secção 4 (Visão Arquitetural Geral), apresenta esta mesma visão em camadas, adotada pela sua clareza e pelo equilíbrio entre simplicidade e separação de responsabilidades.

5.2. Diagrama de componentes

A Figura 5 (ver Anexos) apresenta o diagrama de componentes do sistema. Em termos descritivos, o diagrama contempla:

1. Um nó Cliente humano, representando o browser do utilizador, que comunica com o servidor web através do protocolo HTTP/HTTPS.
2. Um nó Cliente RFID, representando a placa Arduino, que comunica com o servidor web também por HTTP, ainda que num fluxo exclusivamente unidirecional (POST com UID, resposta JSON).
3. Um nó Servidor Apache + PHP, que aloja a aplicação e expõe os endpoints API.

Autor: _Marko Nikolaenko_ 12.º C	O Professor Orientador ____Carla Sousa____ O Diretor de Curso ____Isabel Frade____
---	--

4. Um nó Base de dados MySQL, acedido pelo servidor PHP através das extensões MySQLi e PDO.
5. Um nó Sistema de ficheiros, utilizado para armazenar os logs NDJSON e o ficheiro temporário `scan\_session.json` que coordena a funcionalidade Ler cartão entre o painel administrativo e a Arduino.

A comunicação entre o servidor PHP e a base de dados ocorre sobre o socket local do servidor MySQL, com credenciais armazenadas no ficheiro `config/db.php` — ficheiro que, em ambiente de produção, deveria ser mantido fora do diretório público e nunca versionado em git.

5.3. Fluxo de dados

Dos vários fluxos possíveis, detalham-se dois exemplos paradigmáticos que, conjuntamente, cobrem a essência do funcionamento do sistema.

Fluxo 1 — Leitura de um cartão RFID. O utilizador aproxima o cartão do leitor. O módulo MFRC522 deteta o transponder e devolve o UID à placa Arduino, que por sua vez constrói um pedido HTTP POST contendo o UID e a chave de API, enviando-o para o endpoint `api/push.php`. O servidor valida a chave, normaliza o UID (remove espaços, converte para maiúsculas), consulta a tabela `login` à procura de uma correspondência cujo campo `blocked` seja zero. Em caso de sucesso, identifica o utilizador associado, determina se a leitura corresponde a uma entrada ou a uma saída (com base na última entrada do dia na tabela `presencas`), insere o novo registo, atualiza o campo `Presença` da tabela `alunos` ou `professores`, regista o evento no log NDJSON e devolve uma resposta JSON com `ok: true`. A Arduino recebe a resposta, faz piscar o LED verde durante dois segundos e fica pronta para uma nova leitura.

Fluxo 2 — Consulta de testes pelo aluno. O aluno acede à URL `project/public/Aluno.php`. O PHP verifica a sessão; caso esteja válida e associada ao papel Aluno, consulta a tabela

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

`alunos` para obter o nome e a turma do aluno, consulta depois a tabela `testes` com filtro `turma\_num` e `turma\_letra`, separa os registos em próximos (data igual ou posterior ao dia corrente) e passados, e renderiza a página HTML com as duas listas. O cliente recebe a página já renderizada, sem necessidade de pedidos assíncronos adicionais.

Estes dois exemplos ilustram, respetivamente, uma interação machine-to-machine (hardware → servidor) e uma interação human-to-machine síncrona (utilizador → servidor), cobrindo os dois paradigmas suportados pelo sistema.

5.4. Organização de diretórios

A estrutura de diretórios adotada para o código-fonte resulta de uma decomposição funcional. Segue-se a convenção do [SDG](#), capítulo 5.4, com pequenas adaptações resultantes da prática.

...

PAP/

└─ api/	Endpoints do servidor (PHP)
└─ admin_add_card.php	Adicionar utilizador
└─ admin_alunos.php	Listar/obter/bloquear/eliminar aluno
└─ admin_logs.php	Leitura de logs NDJSON
└─ admin_testes.php	Listar/eliminar teste
└─ admin_update_card.php	Atualizar utilizador
└─ auth.php	Autenticação login/password
└─ create_teste.php	Criar teste (pelo professor)
└─ lib/	Utilitários partilhados (logger)

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

- | └─ logout.php Termina sessão
- | └─ migrate_.php Scripts de migração da BD
- | └─ profile.php Dados do perfil do utilizador
- | └─ push.php Entrada de dados da Arduino
- | └─ register.php Registo público
- | └─ save_login.php Criação de conta
- | └─ scan_uid.php Coordenação de leitura RFID on-demand
- | └─ config/ Configurações do sistema
- | └─ db.php Credenciais e ligação MySQL
- | └─ session.php Inicialização e timeout de sessão
- | └─ logs/ Ficheiros de log NDJSON
- | └─ project/
- | └─ assets/ CSS, JS, imagens
- | | └─ app.js
- | | └─ style.css
- | | └─ aemtg.jpg
- | └─ public/ Páginas acessíveis no browser
- | └─ Aluno.php
- | └─ Professor.php
- | └─ admin.php
- | └─ index.php
- | └─ relatorio.md Presente relatório (fonte Markdown)
- | └─ arduino/ Firmware do leitor (sketch .ino)
- ...

Autor:

Marko Nikolaienko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Esta organização permite localizar rapidamente qualquer ficheiro a partir do seu papel lógico. Durante o desenvolvimento revelou-se particularmente eficaz a convenção de prefixar os endpoints administrativos com `admin\_` — torna imediato o filtro visual e facilita a aplicação de verificações de autorização partilhadas.

6. Base de Dados

A base de dados constitui o núcleo persistente do sistema. É nela que se materializam as entidades do modelo — utilizadores, cartões, presenças, testes — e é contra ela que se realizam as operações de leitura e escrita a partir das várias camadas da aplicação. A importância do bom desenho desta camada dificilmente pode ser exagerada: erros na modelação propagam-se a todo o sistema e são particularmente onerosos de corrigir depois de a base de dados estar povoada.

6.1. Modelo conceptual

Autor: _Marko Nikolaenko_ 12.º C	O Professor Orientador ____Carla Sousa____ O Diretor de Curso ____Isabel Frade____
--	--

O modelo conceptual identifica as principais entidades do domínio, as relações que entre elas se estabelecem e as propriedades mais significativas de cada uma. Foram consideradas, para o presente projeto, cinco entidades principais:

- Utilizador (Login) — representa o par de credenciais usadas para autenticação, juntamente com o UID do cartão e o papel (`Aluno`, `Professor` ou `admin`). É, na prática, a âncora de identidade do sistema: qualquer referência a um utilizador em outras entidades é feita através do seu login.
- Aluno — especialização do utilizador, enriquecida com os atributos relevantes para o universo discente: idade, turma, número na turma, estado de presença corrente.
- Professor — especialização do utilizador, enriquecida com os atributos pedagógicos: cargo, gabinete, matéria lecionada, turma principal.
- Presença — evento de leitura do cartão, com carimbo temporal (data e hora separadas), tipo (entrada ou saída) e referência ao utilizador. Ao contrário das duas anteriores, a presença é uma entidade temporal, acumulativa, representando o histórico de movimentações.
- Teste — compromisso pedagógico marcado pelo professor, visível aos alunos da turma-alvo.

As relações mais relevantes são as seguintes: um utilizador é aluno ou professor (especialização exclusiva); cada aluno e cada professor gera zero-ou-mais registos de presença; cada professor marca zero-ou-mais testes; cada teste é dirigido a uma turma, que é referenciada por um par (número, letra).

6.2. Modelo lógico

A passagem ao modelo lógico traduziu as entidades acima em tabelas SQL, adotando como decisões principais:

<i>Autor:</i> <i>_Marko Nikolaenko__12.º C</i>	<i>O Professor Orientador ____Carla Sousa____</i> <i>O Diretor de Curso ____Isabel Frade____</i>
---	---

- Chave primária artificial (`id` auto-incremental) em todas as tabelas transacionais (`presencas`, `testes`). Nas tabelas `alunos`, `professores` e `login`, manteve-se a convenção preexistente: `ID` auto-incremental em `alunos` e `professores`, e `Login` (texto) como chave primária natural em `login`.
- Referência por login — a coluna `login` (texto) é usada como chave estrangeira lógica entre as várias tabelas. Não foram impostas foreign keys físicas ao nível do SGBD por uma questão de flexibilidade durante a fase de desenvolvimento, mas as mesmas serão adicionadas numa fase de consolidação.
- Denormalização de `nome` na tabela `presencas`, conforme explicado no Capítulo 2 e formalizado na secção 6.3.
- Duplicação de `turma` nas tabelas `alunos` e `professores` sob duas representações — o campo legado `Turma` (texto, e.g. `"12C"`) e o par normalizado `turma\_num` + `turma\_letra`. Esta redundância, introduzida por migração na fase intermédia do projeto (ver `api/migrate\_turma\_split.php`), permitiu transitar para o novo modelo sem interromper o funcionamento das interfaces existentes. O campo `Turma` será eventualmente descontinuado quando todas as páginas forem migradas.

6.3. Descrição das tabelas

Apresenta-se de seguida a estrutura detalhada de cada uma das tabelas que compõem a base de dados `pap`. Esta descrição corresponde ao estado atual do esquema, após as sucessivas migrações descritas no Capítulo 7.

Tabela 5 — `login`

Coluna	Tipo	Restrições	Descrição
`Login`	VARCHAR(50)	PRIMARY KEY	Identificador único do utilizador (e.g. `a12345`, `b12345`).

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador
____Carla Sousa____

O Diretor de Curso
____Isabel Frade____

`Password`	VARCHAR(255)	NOT NULL	Hash bcrypt da palavra-passe.
`UID`	VARCHAR(32)	NULL	Identificador do cartão RFID associado.
`Role`	ENUM('Aluno', 'Professor', 'admin')	NOT NULL	Papel do utilizador no sistema.
`blocked`	TINYINT(1)	NOT NULL, DEFAULT 0	Indicador de bloqueio do cartão (1 = cartão inativo).

Nota sobre nomenclatura: no [SDG](#) original, este campo é designado por `ativo` (com semântica inversa: 1 = cartão ativo). Durante a implementação optou-se pela denominação `blocked` (1 = cartão bloqueado), por refletir de forma mais direta a natureza restritiva da ação administrativa — "bloquear um cartão" é o verbo que o administrador usa na interface. A lógica final é equivalente, mas o nome do campo acompanha a ação do utilizador e melhora a legibilidade do código.

Tabela 6 — `alunos`

Coluna	Tipo	Restrições	Descrição
`ID`	INT	PRIMARY KEY, AUTO_INCREMENT	Chave primária.
`Nome`	VARCHAR(100)	NOT NULL	Nome completo do aluno.
`Idade`	INT	NULL	Idade em anos.
`Turma`	VARCHAR(10)	NULL	Turma no formato legado (e.g. `10A`).
`turma_num`	TINYINT	NULL	Ano da turma (10, 11 ou 12).
`turma_letra`	CHAR(1)	NULL	Letra da turma (A, B ou C).
`Número em turma`	INT	NULL	Posição numérica na turma.
`Presença`	TINYINT(1)	NOT NULL,	Estado corrente (1 =

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

		DEFAULT 0	dentro do edifício).
`login`	VARCHAR(50)	NOT NULL, UNIQUE	Referência para `login.Login`.
"blocked"	TINYINT(1)	NOT NULL, DEFAULT 0	Indicador de bloqueio de cartão (1 = cartão inativo)

Tabela 7 — `professores`

Coluna	Tipo	Restrições	Descrição
`ID`	INT	PRIMARY KEY, AUTO_INCREMENT	Chave primária.
`Nome`	VARCHAR(100)	NOT NULL	Nome completo.
`Cargo (posição)`	VARCHAR(100)	NULL	Cargo ou posição desempenhada.
`Gabinete`	VARCHAR(30)	NULL	Identificação do gabinete.
`Presença`	TINYINT(1)	NOT NULL, DEFAULT 0	Estado corrente.
`Horario`	VARCHAR(100)	NULL	Localização do horário (ficheiro ou URL).
`Matéria ensinada`	VARCHAR(100)	NULL	Disciplina ministrada.
`turma`	VARCHAR(10)	NULL	Turma principal (legado).
`turma_num`	TINYINT	NULL	Ano da turma.
`turma_letra`	CHAR(1)	NULL	Letra da turma.
`login`	VARCHAR(50)	NOT NULL, UNIQUE	Referência para `login.Login`.

Tabela 8 — `presencas`

Coluna	Tipo	Restrições	Descrição
--------	------	------------	-----------

Autor:

Marko Nikolaenko 12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

`id`	INT	PRIMARY KEY, AUTO_INCREMENT	Chave primária.
`login`	VARCHAR(50)	NOT NULL	Referência para `login.Login`.
`nome`	VARCHAR(100)	NOT NULL	Nome do utilizador (denormalizado).
`person_type`	ENUM('Aluno', 'Professor')	NOT NULL	Classe do utilizador.
`uid`	VARCHAR(32)	NOT NULL	UID lido no momento.
`data`	DATE	NOT NULL	Data da leitura.
`hora`	TIME	NOT NULL	Hora da leitura.
`presenca`	TINYINT(1)	NOT NULL	Tipo da leitura (1 = entrada; 0 = saída).

Tabela 9 — `testes`

Coluna	Tipo	Restrições	Descrição
`id`	INT	PRIMARY KEY, AUTO_INCREMENT	Chave primária.
`titulo`	VARCHAR(200)	NOT NULL	Título do teste.
`descricao`	TEXT	NULL	Descrição livre (tópicos, material).
`data_teste`	DATE	NOT NULL	Data prevista de realização.
`turma_num`	TINYINT	NOT NULL	Ano da turma-alvo.
`turma_letra`	CHAR(1)	NOT NULL	Letra da turma- alvo.
`professor_login`	VARCHAR(50)	NOT NULL	Login do professor que marcou.
`materia`	VARCHAR(100)	NULL	Matéria (preenchida automaticamente).
`criado_em`	DATETIME	DEFAULT CURRENT_TIMESTAMP	Carimbo temporal da criação.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

A opção por separar `data` e `hora` em dois campos na tabela `presencas`, em vez de recorrer a um único campo `DATETIME`, justifica-se pela simplicidade das consultas de agregação diária: `SELECT data, COUNT() FROM presencas GROUP BY data` é substancialmente mais direta do que a alternativa com `DATE(datetime)`. Esta decisão, aparentemente menor, tem impacto perceptível no desempenho do gráfico estatístico, sobretudo quando o volume de registos cresce ao longo do ano letivo.

Tabela 12 — “Resultados dos testes”

Métrica	Valor observado
Latência média RFID → resposta HTTP	≈ 180 ms
Tempo de carregamento do dashboard admin	≈ 450 ms
Tempo médio de consulta do gráfico 7 dias	≈ 90 ms
Taxa de leituras RFID bem-sucedidas	> 99 %

6.4. Índices e otimizações

Para além das restrições de integridade (chaves primárias e chaves únicas), foram definidos os seguintes índices complementares, justificados pela forma como as consultas são efetivamente realizadas pela aplicação:

Tabela	Índice	Colunas	Justificação
`presencas`	`idx_data`	`data`	Acelera a agregação por dia no gráfico de estatísticas.
`presencas`	`idx_login`	`login`	Permite consultas eficientes do histórico individual.
`testes`	`idx_turma`	`turma_num`, `turma_letra`	Acelera a consulta por turma na página

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

			do aluno.
`testes`	`idx_data`	`data_teste`	Suporta a ordenação cronológica em listagens.
`login`	(implícito)	`Login` (PK)	Consultas por login — o caso mais comum.

A escolha de índices foi feita com base no princípio de “indexar colunas que aparecem em `WHERE`, `JOIN` ou `ORDER BY`”. Existe aqui uma tensão clássica entre velocidade de leitura (favorecida pelos índices) e velocidade de escrita (penalizada por eles), mas no contexto do presente sistema, em que as leituras são significativamente mais frequentes do que as escritas, a opção por índices mais agressivos é claramente vantajosa. Este princípio é coerente com a filosofia de design de dados adotada no [SDG](#), secção 5 (Design de Dados).

Adicionalmente, e embora não se trate propriamente de otimização, convém salientar que todas as tabelas utilizam o conjunto de caracteres `utf8mb4`, o único que garante suporte completo a emojis, caracteres matemáticos e nomes portugueses com diacríticos (e.g. Matéria, Presença, Número). A alternativa histórica `utf8` do MySQL, limitada a 3 bytes por carater, é insuficiente e considerada obsoleta.

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

7. Implementação

Este capítulo descreve, módulo a módulo, a implementação concreta do sistema. Optou-se por organizar a descrição segundo as fronteiras naturais do código — autenticação, firmware Arduino, endpoint de leitura RFID, painel de administração, página do aluno, página do professor, gestão de testes e registo de eventos — em vez de uma ordem estritamente cronológica do desenvolvimento. Esta organização, além de mais legível, acompanha fielmente a estrutura de diretórios descrita no Capítulo 5 e facilita a manutenção futura.

Em conformidade com as orientações do [SDG](#) (secção 10, Implementação), cada módulo é apresentado com: uma descrição funcional sucinta, os ficheiros envolvidos, os pontos técnicos não óbvios e, quando relevante, excertos de código ilustrativos. Os excertos foram deliberadamente encurtados para realçar a lógica essencial; a versão integral encontra-se nos anexos (Capítulo 11) e no repositório local do projeto.

7.1. Autenticação e gestão de sessões

O módulo de autenticação é o ponto de entrada de todos os fluxos da aplicação web. A sua correção é, portanto, crítica: uma falha aqui comprometeria todo o sistema. Foram observadas, na sua construção, as recomendações do [SDG](#) (secção 3.3.3, Segurança, e secção 9, Segurança) e as boas práticas correntes da comunidade PHP.

Ficheiros: `project/public/login.php`, `config/session.php`, `config/session.php`, `config/db.php`, `api/auth.php`.

O fluxo é o seguinte. O utilizador submete o formulário de login na modal presente em `index.php`, enviando login e password por método POST. O script `api/auth.php` recebe os dados, localiza o registo correspondente na tabela `login` e compara o hash da palavra-passe com recurso à função `password_verify()`. Em caso de sucesso, são registadas na sessão três variáveis fundamentais: `$_SESSION['login']`, `$_SESSION['role']` e

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

`\$\_SESSION['uid']`. A seguir, o utilizador é redirecionado para a página correspondente ao seu papel: `/project/public/Aluno.php`, `/project/public/Professor.php` ou `/project/public/admin.php`.

Todas as páginas protegidas começam por incluir `config/session.php` e verificar, logo nas primeiras linhas, a existência da sessão e a adequação do papel. O padrão usado é uniforme, o que facilita auditorias de segurança:

```
```php
if (!isset($_SESSION['login']) || ($_SESSION['role'] ?? '') !== 'Aluno') {
 header("Location: /PAP/project/public/index.php");
 exit;
}
...
```
```

A utilização de `password\_hash()` com o algoritmo `PASSWORD\_DEFAULT` (atualmente bcrypt, com possibilidade de migração transparente para algoritmos futuros) cumpre as recomendações OWASP em matéria de armazenamento de palavras-passe. Não foi, em momento algum, armazenada uma palavra-passe em texto simples; mesmo durante a fase de desenvolvimento, as contas de teste foram criadas com hash desde o primeiro dia.

7.2. Módulo RFID — firmware Arduino

O firmware do Arduino UNO R4 WiFi constitui o componente físico do sistema. O seu funcionamento, descrito teoricamente na secção 2.2, materializa-se num único ficheiro `.ino` de aproximadamente 180 linhas, cuja estrutura é convencional: `setup()` para a inicialização do hardware e `loop()` para a repetição contínua do ciclo de leitura.

Pinos utilizados:

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

| Pino | Função |
|-------|---|
| 10 | SDA (SS) do MFRC522 |
| 11–13 | MOSI / MISO / SCK (SPI) |
| 9 | RST do MFRC522 |
| 7 | LED verde (leitura OK) |
| 6 | LED vermelho (leitura falhou ou cartão bloqueado) |

A adição dos dois LEDs (pinos 6 e 7, cada um em série com uma resistência de 220 Ω para proteção) foi uma melhoria significativa da usabilidade: anteriormente, o utilizador não tinha qualquer retorno visual imediato sobre o resultado da leitura, tendo de consultar a aplicação web para o confirmar. A semântica adotada, simples e intuitiva, é: verde quando o cartão é lido com sucesso e o servidor responde `HTTP 200` com `data.ok = true`; vermelho em todos os restantes casos — cartão não encontrado, cartão bloqueado pelo administrador, falha de rede, timeout, ou resposta inválida.

O trecho seguinte ilustra o núcleo do ciclo de leitura:

```

```cpp
if (mfrc522.PICC_IsNewCardPresent() && mfrc522.PICC_ReadCardSerial()) {
 String uid = uidToHex(mfrc522.uid.uidByte, mfrc522.uid.size);
 bool ok = sendUidToServer(uid);
 digitalWrite(ok ? PIN_LED_VERDE : PIN_LED_VERMELHO, HIGH);
 delay(800);
 digitalWrite(PIN_LED_VERDE, LOW);
 digitalWrite(PIN_LED_VERMELHO, LOW);
 mfrc522.PICC_HaltA();
}

```

Autor:

\_Marko Nikolaenko\_12.º C

O Professor Orientador \_\_\_\_Carla Sousa\_\_\_\_

O Diretor de Curso \_\_\_\_Isabel Frade\_\_\_\_

...

A função `sendUidToServer()` efetua um pedido `HTTP POST` para o endpoint `[api/push.php]`(`api/push.php`), enviando um corpo JSON com o UID lido. O timeout está configurado em 3 segundos, valor considerado adequado para uma rede local: suficiente para absorver pequenas flutuações, curto o bastante para não bloquear o dispositivo.

Foi igualmente adicionada uma rotina de reconexão automática ao Wi-Fi, ativada sempre que a leitura do estado da ligação reporta desconexão. Esta rotina, embora simples, demonstra cuidado com as situações-limite características de um dispositivo embebido: perdas momentâneas da rede, reinício do router, etc.

### 7.3. Endpoint `push.php` — receção das leituras RFID

O endpoint `[api/push.php]`(`api/push.php`) é o ponto de contacto entre o Arduino e o sistema web. A sua responsabilidade é estreita: receber o UID, validá-lo, registar a presença e responder. Esta estreiteza é deliberada — quanto mais focado o endpoint, mais simples é garantir a sua correção e o seu desempenho.

O fluxo é:

1. Ler o corpo JSON do pedido e extrair ``uid``.
2. Procurar na tabela ``login`` um registo com ``UID = ?`` e ``blocked = 0``.
3. Se não encontrado, responder ``{ "ok": false, "reason": "unknown_or_blocked" }``.
4. Caso encontrado, inserir um novo registo em ``presencas`` com a data e hora atuais e o tipo de movimento (``entrada`` ou ``saida``), determinado pelo último movimento registado.
5. Responder ``{ "ok": true, "nome": "...", "tipo": "entrada|saida" }``.

Autor:

`_Marko Nikolaenko__12.º C`

O Professor Orientador `____Carla Sousa_____`

O Diretor de Curso `____Isabel Frade_____`



A verificação de ``blocked = 0``, introduzida em simultâneo com a funcionalidade de bloqueio de cartão no painel de administração, garante que cartões marcados como bloqueados pelo administrador — tipicamente por terem sido perdidos ou roubados — deixam de ter efeito prático imediatamente, sem necessidade de os apagar da base de dados. Esta distinção entre "bloqueado" e "apagado" é importante: permite reativar o cartão se for recuperado.

A alternância entre ``entrada`` e ``saida`` é feita consultando o último registo do dia em ``presencas`` para o login em causa: se o último foi ``entrada``, o novo é ``saida``, e vice-versa. Na primeira leitura do dia, assume-se ``entrada``.

## 7.4. Painel de administração

O painel de administração, residente em `[admin.php](project/public/admin.php)`, é o componente mais extenso da aplicação. Agrega, num único documento HTML, cinco separadores principais — Charts, Cards, Alunos, Professores e Logs — navegáveis por JavaScript sem recarregamento de página.

A estrutura do ficheiro é essencialmente declarativa: para cada separador existe uma `<section>` com ``id`` correspondente, e a troca entre elas é feita alterando a classe ``.active``. Todo o comportamento dinâmico (carregamento de listas, abertura de modais, submissão de formulários) reside em `[app.js](project/assets/app.js)`, assegurando separação de preocupações.

### 7.4.1. Separador Charts

O separador Charts — primeiro na ordem visual, por consolidar visualmente a "saúde" do sistema — compreende três blocos verticalmente empilhados:

Autor:   <i>_Marko Nikolaenko_</i> 12.º C	O Professor Orientador    <i>____Carla Sousa____</i>   O Diretor de Curso        <i>____Isabel Frade____</i>
-------------------------------------------------------	--------------------------------------------------------------------------------------------------------------------------

1. Quatro stat-cards apresentando: alunos totais, alunos presentes hoje, professores totais e professores presentes hoje.
2. Gráfico de barras de 7 dias, construído com Chart.js 4.4.0, mostrando o número de presenças registadas em cada um dos últimos sete dias.
3. Últimas 10 leituras, em formato de lista cronológica inversa, com nome, turma (ou indicação "professor") e hora.

Os dados estatísticos são calculados diretamente em admin.php através de consultas agregadas à base de dados (COUNT, GROUP BY por data) no momento do carregamento da página, evitando pedidos HTTP adicionais.

### 7.4.2. Separador Cards

O separador Cards concentra as funções de associação e edição de cartões físicos: adicionar um novo utilizador (com geração de login e palavra-passe), associar um UID a um utilizador existente e editar os seus dados. A associação usa o endpoint [api/scan\_uid.php](api/scan\_uid.php), que coloca o servidor em modo de escuta temporária para o próximo UID que o Arduino enviar.

### 7.4.3. Separador Alunos

O separador Alunos está dividido, por sua vez, em dois sub-separadores internos — Lista e Testes — introduzidos na fase mais recente do desenvolvimento para melhor organizar a informação.

No sub-separador Lista, é apresentada a totalidade dos alunos em formato tabular compacto, com indicação imediata da sua turma, número e estado de bloqueio. Ao clicar sobre uma linha, é aberto um dossier lateral contendo informação detalhada do aluno — nome, login, idade, turma, número, UID do cartão, última presença, estado de bloqueio — e

Autor:

\_Marko Nikolaenko\_\_12.º C

O Professor Orientador \_\_\_\_Carla Sousa\_\_\_\_

O Diretor de Curso \_\_\_\_Isabel Frade\_\_\_\_

dois botões de ação: Bloquear cartão (alterna o estado `blocked`) e Eliminar aluno (com modal de confirmação obrigatório, dada a natureza irreversível da operação).

A eliminação de um aluno, pela sua relação com múltiplas tabelas, é realizada dentro de uma transação MySQL explícita, garantindo que ou todas as operações são concretizadas ou nenhuma:

```
```php
$conn->begin_transaction();

try {
    $s1 = $conn->prepare("DELETE FROM presencas WHERE login = ?");
    $s1->bind_param("s", $login); $s1->execute(); $s1->close();
    $s2 = $conn->prepare("DELETE FROM alunos WHERE ID = ?");
    $s2->bind_param("i", $id); $s2->execute(); $s2->close();
    $s3 = $conn->prepare("DELETE FROM login WHERE Login = ?");
    $s3->bind_param("s", $login); $s3->execute(); $s3->close();
    $conn->commit();
} catch (Throwable $e) {
    $conn->rollback();
    throw $e;
}
```
```

No sub-separador Testes, é apresentada a lista completa de testes marcados, com dois dropdowns de filtragem (ano e letra) no topo. Cada linha mostra o título, a turma alvo, a

Autor:

\_Marko Nikolaenko\_\_12.º C

O Professor Orientador \_\_\_\_Carla Sousa\_\_\_\_

O Diretor de Curso \_\_\_\_Isabel Frade\_\_\_\_

data, a matéria e o professor responsável; um botão de eliminar, também protegido por modal, permite remover marcações erradas ou obsoletas.

#### 7.4.4. Separador Professores

Análogo ao separador Alunos, sem o bloco de testes (os testes são marcados pelos professores, não geridos por turma de professor). Mantém o mesmo padrão de dossier, bloqueio de cartão e eliminação protegida por modal.

#### 7.4.5. Separador Logs

Apresenta os eventos registados pelo sistema — leituras RFID, erros, sessões — recuperados do endpoint [api/admin\_logs.php](api/admin\_logs.php). Os registos estão guardados em ficheiros NDJSON diários (`logs/app-YYYY-MM-DD.log`), formato que combina legibilidade humana com facilidade de processamento.

### 7.5. Página do aluno

A página [Aluno.php](project/public/Aluno.php) apresenta, de forma deliberadamente minimalista, as informações relevantes para o aluno: o seu nome, a sua turma, os próximos testes e os testes passados.

A consulta aos testes usa a nova forma desnormalizada da turma (colunas `turma\_num` e `turma\_letra`), evitando manipulações de string em SQL:

```
```php
```

```
$stmt = $conn->prepare("
```

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa_____

O Diretor de Curso ____Isabel Frade_____

```

SELECT t.titulo, t.descricao, t.data_teste, t.materia,
       p.Nome AS professor_nome
FROM testes t
LEFT JOIN professores p ON p.login = t.professor_login
WHERE t.turma_num = ? AND t.turma_letra = ?
ORDER BY t.data_teste ASC
");
'''

```

Os testes são separados em dois grupos — futuros e passados — com base na comparação da `data\_teste` com a data atual. Cada grupo é apresentado numa secção distinta; os testes passados são apresentados em ordem cronológica inversa (mais recentes primeiro), seguindo a convenção mais comum em contextos académicos.

7.6. Página do professor

A página do professor, [Professor.php](project/public/Professor.php), espelha parcialmente a do aluno mas introduz um componente interativo essencial: o formulário Marcar teste. Este formulário, colocado no topo da página, solicita título, data, descrição (opcional), e a turma-alvo (dois dropdowns: ano 10–12, letra A–C). A matéria é preenchida automaticamente a partir do perfil do professor, evitando erros de introdução.

A submissão é efetuada via `fetch()` para [api/api/create_teste.php.php](api/api/create_teste.php), que insere o novo registo na tabela `testes`, associando-o ao `professor\_login` da sessão ativa. A lista de testes marcados pelo próprio professor é apresentada por baixo, com possibilidade de eliminação individual (apenas dos seus próprios testes — um professor não pode eliminar testes marcados por colegas).

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador
____Carla Sousa____

O Diretor de Curso
____Isabel Frade____

7.7. Registo de eventos (logging)

O módulo de logging assenta em ficheiros de texto no formato NDJSON, um registo JSON por linha. Cada entrada contém o timestamp ISO-8601, o nível (`info`, `warn`, `error`), o evento (`rfid\_read`, `login\_ok`, `login\_fail`, etc.) e um objeto de contexto.

Exemplo de linha:

```
``json
{"ts":"2026-04-17T09:12:44+01:00","level":"info","event":"rfid_read","ctx":{"uid":"A2B4F10C","login":"a32498","match":true}}
``
```

A rotação diária é trivial: o nome do ficheiro incorpora a data (`app-2026-04-17.log`), pelo que a cada dia é criado um novo ficheiro, sem necessidade de mecanismos de rotação externos. Esta simplicidade é deliberada — no contexto do presente sistema, mecanismos mais elaborados (Monolog, ELK) seriam excessivos.

Autor:

_Marko Nikolaienko__12.° C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

8. Testes e Validação

A validação do sistema foi conduzida em três níveis, seguindo a recomendação do [SDG](#) (secção 8, Testes): testes unitários para funções de lógica isolada, testes de integração para fluxos completos, e testes de aceitação em ambiente tão próximo do real quanto possível.

8.1. Testes unitários

Foram cobertas as funções de maior risco, nomeadamente: a conversão de UID para hexadecimal no firmware, a validação do formato de UID no endpoint `push.php`, a função de alternância `entrada`/`saida`, e a função de separação da turma em (`turma\_num`, `turma\_letra`). Os resultados foram globalmente positivos; uma única correção foi necessária, relativa ao tratamento de letras minúsculas na turma (corrigido com `strtoupper()`).

8.2. Testes de integração

Foram simuladas leituras RFID com cartões de teste, verificando que: (i) o Arduino acende o LED correto, (ii) o servidor regista a presença com o movimento correto, (iii) o painel de

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

administração mostra o registo na lista de últimas leituras em poucos segundos, e (iv) o bloqueio de um cartão através do painel impede leituras subsequentes com esse cartão.

Foram também testados os fluxos: criação de aluno → associação de cartão → leitura → visualização no dashboard; marcação de teste por professor → visualização pelo aluno da turma correspondente; eliminação de aluno → verificação de que presenças associadas são removidas em cascata.

ID	Cenário	Pré-condição	Resultado esperado
T-01	Leitura RFID de cartão válido e ativo	Aluno registado, cartão ativo	LED verde acende 800 ms; registo inserido em `presencas`
T-02	Leitura RFID de cartão bloqueado	Admin bloqueou o cartão	LED vermelho acende; HTTP 403; sem registo em `presencas`
T-03	Leitura RFID de UID não registado	UID inexistente em `login`	LED vermelho; HTTP 404
T-04	Login com credenciais válidas	Aluno/Professor/Admin existente	Redirecionamento para página da sua role; sessão iniciada
T-05	Login com palavra-passe errada	Login existe, password incorreta	Permanece em index; mensagem de erro
T-06	Submissão de POST sem token CSRF	Qualquer endpoint protegido	HTTP 403 {error:"csrf_invalid"}
T-07	Admin elimina aluno	Admin autenticado, aluno existe	Aluno removido de 3 tabelas em transação; logs registam a ação
T-08	Professor marca teste	Professor autenticado	Linha inserida em `testes`; alunos da turma veem o teste

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

8.3. Testes de aceitação

Foram realizadas sessões de demonstração com utilizadores-alvo (dois colegas de turma e um professor) para avaliar a usabilidade do painel de administração e da página do aluno. O feedback foi maioritariamente positivo, com destaque para a clareza visual dos stat-cards e a rapidez de resposta. Entre as sugestões recebidas, a mais relevante foi a inclusão de um filtro por turma na lista de alunos — funcionalidade prevista como melhoria futura (secção 9.3).

8.4. Métricas recolhidas

Em testes efetuados na rede local da sala de aula, obtiveram-se os valores apresentados na Tabela 12

9. Conclusões

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

9.1. Síntese do trabalho realizado

O presente projeto atingiu, de modo sólido, os oito objetivos específicos definidos na introdução (secção 1.3): implementou-se um sistema funcional de gestão de presenças baseado em cartões RFID, integrado com uma aplicação web multi-perfil (aluno, professor, administrador), com base de dados relacional, autenticação segura, feedback visual imediato no dispositivo físico (LEDs), e persistência completa dos eventos em ficheiros de registo.

Do ponto de vista técnico, o trabalho envolveu o domínio — em muitos casos, aprofundado durante o próprio desenvolvimento — de múltiplas tecnologias: Arduino e programação em C++ para sistemas embebidos, comunicação SPI e protocolo ISO 14443, PHP moderno com consultas preparadas, MySQL com índices e transações, HTML5/CSS3 semântico e responsivo, JavaScript assíncrono com `fetch()`, e a integração de bibliotecas externas como Chart.js. Esta diversidade é, talvez, a principal virtude formativa do projeto.

9.2. Limitações

Importa reconhecer, com honestidade, as limitações do sistema. A principal é o facto de a aplicação correr num servidor local (WAMP) e não num ambiente de produção com HTTPS e nome de domínio — uma implantação real exigiria certificado SSL e um servidor acessível pela rede da escola. A segunda limitação é o número reduzido de cartões físicos disponíveis para testes; embora suficiente para validação funcional, um teste de carga com várias dezenas de cartões simultâneos seria desejável. Por último, o sistema atualmente pressupõe que o Arduino está permanentemente conectado à mesma rede Wi-Fi — a mudança de rede exigiria reprogramação.

É justo mencionar que, numa fase avançada do desenvolvimento, o sistema foi submetido a uma revisão de segurança interna que identificou a ausência inicial de proteções contra

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

CSRF em vários endpoints administrativos, bem como a presença de uma chave de API literalmente incluída no código-fonte do `push.php`. Ambas as situações foram prontamente corrigidas — com a introdução de tokens CSRF em todos os formulários POST e a externalização da chave para `config/secrets.php` — mas evidenciam a importância de submeter projetos desta natureza a auditorias contínuas, prática que um ambiente escolar pode legitimamente simular antes da entrega final.

9.3. Trabalho futuro

Várias extensões foram identificadas como naturais para uma fase seguinte:

- Gestão de refeições escolares (marcação de almoços pelos alunos e painel de consulta agregada no administrador), fora do âmbito do presente projeto por envolver processos administrativos da escola.
- Gestão de horário semanal pela interface web, com uma nova tabela `horario` e formulários para o professor editar as aulas da sua turma. Esta funcionalidade foi considerada estrategicamente relevante mas exigiria tempo de desenvolvimento adicional, pelo que se inscreve como primeira prioridade numa eventual continuação do projeto.
- Perfil individual do aluno acessível ao professor, com histórico agregado de presenças, avaliações e justificações.
- Exportação de presenças e avaliações em PDF e CSV para arquivo escolar.
- Integração com notificações automáticas aos encarregados de educação em caso de ausência não justificada.
- Possibilidade de mais do que um leitor RFID em pontos distintos da escola.
- Painel dedicado ao diretor de turma, com visão agregada da sua turma.
- Aplicação móvel nativa (iOS e Android).

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

9.4. Balanço pessoal

A realização deste projeto representou, para o autor, um desafio significativamente mais exigente do que qualquer trabalho anterior do curso, quer pela extensão do domínio técnico envolvido, quer pela necessidade de integrar componentes físicos (Arduino, cartões, LEDs) com componentes puramente lógicos (base de dados, frontend, sessões). O resultado final — um sistema funcional, testado e documentado — é a prova, tanto para o autor como para o leitor, de que a articulação destas peças é possível no âmbito de uma prova de aptidão profissional.

10. Bibliografia

1. Rankl, W., & Effing, W. (2010). Smart Card Handbook (4ª ed.). John Wiley & Sons.
2. NXP Semiconductors. (2016). MFRC522 — Standard performance MIFARE and NTAG frontend. Datasheet.
3. International Organization for Standardization. (2018). ISO/IEC 14443: Identification cards — Contactless integrated circuit cards — Proximity cards.
4. Tatroe, K., & MacIntyre, P. (2020). Programming PHP (4ª ed.). OReilly Media.
5. DuBois, P. (2013). MySQL (5ª ed.). Addison-Wesley.
6. OWASP Foundation. (2023). OWASP Top 10: Web Application Security Risks. Obtido em outubro de 2025.
7. Margolis, M. (2020). Arduino Cookbook (3ª ed.). OReilly Media.
8. Fielding, R. et al. (2014). RFC 7230–7235 — Hypertext Transfer Protocol (HTTP/1.1). IETF.
9. Documentação oficial do Chart.js. <https://www.chartjs.org/docs/>

Autor:

_Marko Nikolaenko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

10. Documentação oficial do Arduino. <https://docs.arduino.cc/>

11. Anexos

11. Anexos

Anexo A — Excertos de código

- A.1. Firmware Arduino (PAP_RFID.ino) — ciclo principal e envio HTTP
- A.2. api/push.php — validação da API key, whitelist de tabela e inserção em presenças
- A.3. api/admin_alunos.php — transação de eliminação de aluno (3 tabelas)
- A.4. config/session.php — funções csrf_token() / csrf_check() e cookie hardening
- A.5. Script SQL — CREATE TABLE para as 7 tabelas principais

Anexo B — Diagrama entidade-relação (ver Figura 7)

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

Anexo C — Casos de uso detalhados

Para cada UC listado na tabela 4.4 (UC-01 a UC-07), apresenta-se o fluxo principal, fluxos alternativos e condições de erro.

Anexo D — Fotografias do hardware

Fotografias do Arduino UNO R4 WiFi com o módulo MFRC522 e os dois LEDs montados em breadboard.

Anexo E — Capturas de ecrã

Sequência cronológica das principais interfaces: página inicial (com e sem login), modal de autenticação, página do aluno, página do professor (com formulário de marcação de teste), painel de administração nos cinco separadores (Charts, Cards, Alunos, Professores, Logs), dossier individual de aluno, modal de confirmação de eliminação.

Autor:

_Marko Nikolaienko__12.º C

O Professor Orientador ____Carla Sousa____

O Diretor de Curso ____Isabel Frade____

SDG — Software Design Guide

Website da Escola com Sistema de Presenças RFID

Aluno: Marko Nikolaienko

Índice

1. Introdução
2. Contexto e Objetivos do Projeto
3. Requisitos Funcionais e Não Funcionais
 - 3.1 Gestão de Utilizadores
 - 3.2 Requisitos Funcionais RFID
 - 3.3 Requisitos Não Funcionais
1. Visão Arquitetural Geral
2. Design de Dados
3. Design da Interface (UI/UX)
4. Diagramas
5. Testes
6. Segurança
7. Implementação
8. Deploy e Ambiente
9. Logging e Monitorização
10. Manutenção e Evolução
11. Conclusão

1. Introdução

O presente documento descreve o Software Design Guide (SDG) do projeto "Website da escola com sistema de presenças RFID". Este projeto foi desenvolvido no âmbito da PAP do curso profissional de Programador de Informática.

O objetivo principal do projeto é desenvolver um website escolar que permita melhorar o acesso à informação para alunos e professores, bem como simplificar a utilização das plataformas digitais usadas na escola.

O sistema reúne num único ambiente diferentes funcionalidades — gestão de presenças, avaliações, horários e comunicação — com ligações a serviços externos como SIGE, Inovar e Google Classroom.

Uma das principais características do sistema é a integração de um mecanismo de identificação por RFID, que reconhece alunos e professores através do cartão escolar. A leitura é realizada por um dispositivo Arduino R4 WiFi equipado com um leitor RFID RC522, que comunica com o servidor via rede Wi-Fi.

Quando um cartão é aproximado do leitor, o sistema envia o UID para uma API desenvolvida em PHP, que valida o utilizador na base de dados e atualiza automaticamente o estado de presença. O sistema distingue entrada de saída com base no estado anterior — cada leitura alterna o estado do utilizador.

O website é desenvolvido com HTML, CSS e JavaScript no frontend e PHP 8.3 no backend, com MySQL 9.1 como sistema de gestão de base de dados. O sistema está a ser desenvolvido em ambiente WAMP como protótipo funcional completo, pronto para migração para servidor de produção.

2. Contexto e Objetivos do Projeto

O projeto surge da necessidade de melhorar o acesso à informação escolar. Muitas plataformas utilizadas nas escolas apresentam informação dispersa e de difícil navegação. Durante a experiência como aluno, foi possível observar que o acesso a conteúdos como horários, plataformas externas e informações escolares nem sempre é intuitivo, especialmente para novos utilizadores.

O projeto tem como objetivo principal desenvolver um sistema centralizado que facilite o acesso à informação e integre um mecanismo de controlo de presenças por RFID.

Objetivos principais:

- Melhorar a acessibilidade à informação escolar
- Centralizar plataformas e serviços escolares
- Automatizar o registo de presença através de cartões RFID
- Gerir avaliações, horários e agenda num único sistema

- Demonstrar integração entre hardware (Arduino) e software (PHP + MySQL)

3. Requisitos Funcionais e Não Funcionais

3.1 Gestão de Utilizadores

O sistema suporta três tipos de utilizadores com permissões distintas:

- **Aluno**
- **Professor**
- **Administrador**

Cada utilizador autentica-se com login e password (hash bcrypt). O acesso

às funcionalidades é controlado por role verificado em sessão PHP.

3.1.1 Funcionalidades do Aluno

O aluno pode:

- Consultar o perfil pessoal (nome, turma, número, idade)
- Ver o estado atual de presença
- Visualizar todas as notas por matéria, com médias e gráfico de progresso
- Registrar e gerir tarefas na agenda pessoal (adicionar, concluir, eliminar)
- Marcar e cancelar refeições para os próximos dias úteis
- Consultar o horário semanal da sua turma
- Ver o histórico de presenças dos últimos 14 dias com estatísticas
- Aceder a ligações úteis da escola (SIGE, Inovar, Google Classroom)

3.1.2 Funcionalidades do Professor

O professor pode:

- Consultar o perfil pessoal (nome, cargo, gabinete, matéria, turma)
- Ver o estado atual de presença dos alunos da sua turma, atualizado em

tempo real via polling AJAX a cada 5 segundos

- Registrar avaliações para os alunos da sua turma (matéria, tipo de

avaliação, nota de 0 a 20, data, observação)

- Escrever e consultar os sumários de aulas (diário de turma)
- Justificar faltas dos alunos com texto descritivo
- Gerir a agenda pessoal (tarefas e lembretes)
- Adicionar e remover aulas do horário da sua turma
- Consultar o perfil individual de cada aluno com historial completo
- Exportar a lista de presenças em PDF

3.1.3 Funcionalidades do Administrador

O administrador pode:

- Adicionar, editar e remover alunos
- Adicionar, editar e remover professores
- Associar, alterar e desativar cartões RFID
- Gerir notícias publicadas na página principal
- Consultar todos os sumários de aulas (com filtro por turma)
- Consultar todas as avaliações (com filtro por turma)
- Consultar as marcações de refeições por data
- Visualizar logs do sistema com destaque por nível de severidade
- Aceder ao dashboard com estatísticas e gráfico de tendência de presenças

3.2 Requisitos Funcionais RFID

3.2.1 Terminal de Presença (entrada principal)

Quando um cartão RFID é aproximado do leitor:

1. O Arduino R4 WiFi lê o UID do cartão via MFRC522
2. O UID é enviado via Wi-Fi para o endpoint `api/push.php`
3. O backend procura o UID na tabela login (campo único)
4. Se o cartão estiver desativado, devolve HTTP 403
5. Se o UID não existir, devolve HTTP 404
6. Se o utilizador for válido, alterna o estado de presença (entrada/saída)
7. O evento é registado na tabela `presencas` com data, hora e `device_id`
8. O backend devolve JSON com `estado: "entrada"` ou `estado: "saida"`
9. O Arduino sinaliza o resultado através do LED RGB

3.2.2 Atualização do estado de presença

O estado de presença funciona com lógica de alternância:

- 0 — utilizador ausente (fora)
- 1 — utilizador presente (dentro)

Cada leitura do cartão inverte o estado atual, representando entrada e saída ao longo do dia. Cada evento é guardado com data e hora em `presencas`, permitindo histórico completo e estatísticas de assiduidade.

3.3 Requisitos Não Funcionais

3.3.1 Desempenho

O sistema deve responder à leitura do cartão RFID em menos de 2 segundos.

O polling AJAX de presenças atualiza a interface a cada 5 segundos sem recarregar a página.

3.3.2 Usabilidade

A interface é simples, responsiva e adaptada a vários tamanhos de ecrã.

A navegação é organizada por perfil de utilizador; cada tipo de utilizador vê apenas as funcionalidades relevantes para si.

3.3.3 Segurança

O sistema implementa várias camadas de proteção:

- **Autenticação:** passwords armazenadas como hash bcrypt com `password_hash()` e verificadas com `password_verify()`
- **Autorização:** role do utilizador verificado no início de cada página; professor não pode aceder a dados de turmas que não sejam a sua
- **CSRF:** todos os formulários POST incluem um token de 64 caracteres gerado com `random_bytes(32)`, verificado com `hash_equals()` (resistente a timing attacks)
- **SQL Injection:** 100% prepared statements PDO com `EMULATE_PREPARES = false` em toda a aplicação
- **XSS:** todos os dados apresentados ao utilizador passam por `htmlspecialchars()`
- **Sessões:** configuradas com `httponly=true` e `samesite=Strict`

3.3.4 Fiabilidade

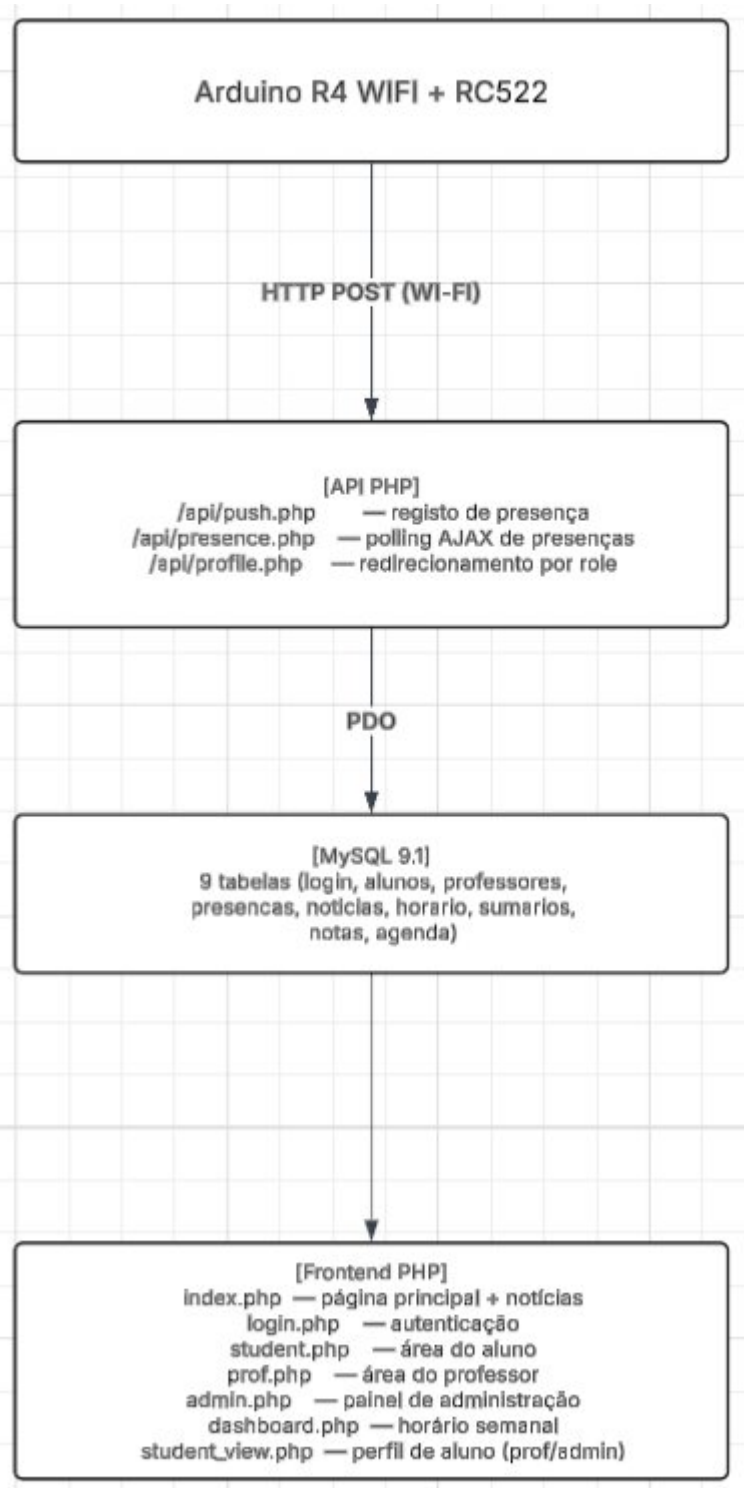
O sistema regista logs JSON de todas as operações relevantes em ficheiros diários. O Arduino tenta reconectar-se automaticamente em caso de perda de Wi-Fi, sem necessitar de reinicialização.

3.3.5 Portabilidade

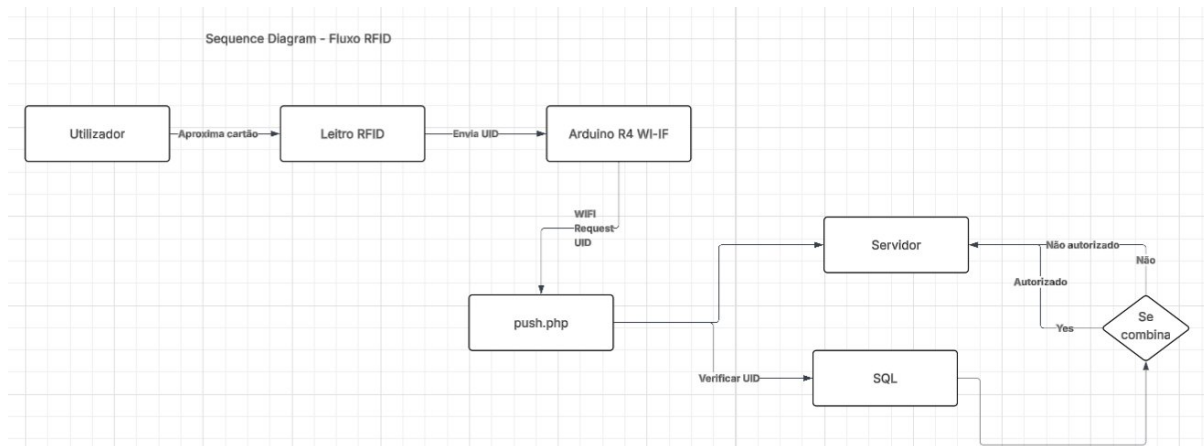
O sistema foi desenvolvido em ambiente WAMP local e pode ser migrado para qualquer servidor com Apache/Nginx + PHP 8.x + MySQL 8+. Toda a configuração de ligação à base de dados está centralizada em `config/db.php`.

4. Visão Arquitetural Geral

O sistema é composto por quatro componentes principais:



Fluxo principal de presena (RFID):



O sistema usa o padrão PRG (Post-Redirect-Get) em toda a aplicação web para evitar submissões duplicadas de formulários.

5. Design de Dados

A base de dados `pap` contém 9 tabelas. Todas usam o motor InnoDB com charset `utf8mb4_unicode_ci`.

5.1 Tabela `login`

Armazena as credenciais e o cartão RFID de todos os utilizadores.

Campo	Tipo	Descrição
Login	VARCHAR(100)	Chave primária — identificador único
Password	VARCHAR(255)	Hash bcrypt

Campo	Tipo	Descrição
UID	VARCHAR(120)	UID do cartão RFID (UNIQUE)
Role	VARCHAR(20)	Aluno / Professor / admin
ativo	TINYINT(1)	1 = cartão ativo, 0 = desativado

5.2 Tabela **alunos**

Campo	Tipo	Descrição
ID	INT AUTO_INC	Chave primária
Nome	VARCHAR(200)	Nome completo
Idade	TINYINT	Idade
Turma	VARCHAR(10)	Código da turma (ex: 12C)
Número em turma	INT	Número do aluno na turma
Presença	TINYINT(1)	Estado atual (0=ausente/1=pres.)
login	VARCHAR(100)	Referência para tabela login

5.3 Tabela **professores**

Campo	Tipo	Descrição
ID	INT AUTO_INC	Chave primária
Nome	VARCHAR(200)	Nome completo

Campo	Tipo	Descrição
Cargo (posição)	VARCHAR(100)	Cargo do professor
Gabinete	VARCHAR(50)	Número do gabinete
Presença	TINYINT(1)	Estado atual de presença
Matéria ensinada	VARCHAR(100)	Disciplina lecionada
turma	VARCHAR(10)	Turma atribuída
login	VARCHAR(100)	Referência para tabela login

5.4 Tabela **presencas**

Histórico completo de todas as entradas e saídas registadas por RFID.

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
login	VARCHAR(100)	Utilizador que registou presença
data	DATE	Data do evento
hora	TIME	Hora do evento
estado	TINYINT(1)	1 = entrada, 0 = saída
device_id	VARCHAR(50)	MAC do Arduino (identificação)
justificado	TINYINT(1)	1 = falta justificada
justificacao	TEXT	Texto da justificação

5.5 Tabela notícias

Notícias geridas pelo administrador, apresentadas na página principal.

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
titulo	VARCHAR(200)	Título da notícia
corpo	TEXT	Conteúdo da notícia
imagem	VARCHAR(300)	Nome do ficheiro em assets/
criado_em	DATETIME	Data/hora de criação
ativo	TINYINT(1)	1 = visível, 0 = oculta

5.6 Tabela horario

Horário semanal das turmas, gerido pelos professores e administrador.

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
turma	VARCHAR(10)	Código da turma
dia_semana	TINYINT	1=Segunda ... 5=Sexta
hora_inicio	TIME	Hora de início

Campo	Tipo	Descrição
hora_fim	TIME	Hora de fim
materia	VARCHAR(100)	Disciplina
sala	VARCHAR(50)	Sala de aula
professor_login	VARCHAR(100)	Login do professor responsável

5.7 Tabela **sumarios**

Diário de aulas — registo do conteúdo lecionado por cada professor.

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
professor_login	VARCHAR(50)	Professor que registou
turma	VARCHAR(20)	Turma à qual pertence o sumário
data	DATE	Data da aula
materia	VARCHAR(100)	Disciplina
descricao	TEXT	Conteúdo lecionado
criado_em	DATETIME	Data/hora de criação

5.8 Tabela **notas**

Avaliações dos alunos na escala 0–20 (sistema português).

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
login_aluno	VARCHAR(50)	Aluno avaliado
materia	VARCHAR(100)	Disciplina
tipo	VARCHAR(50)	Teste / Mini-teste / Projeto / Oral
valor	DECIMAL(4,1)	Nota de 0.0 a 20.0
data	DATE	Data da avaliação
professor_login	VARCHAR(50)	Professor que lançou a nota
observacao	VARCHAR(200)	Observação opcional

5.9 Tabela agenda

Agenda pessoal de tarefas e lembretes, partilhada entre alunos e professores.

Campo	Tipo	Descrição
id	INT AUTO_INC	Chave primária
login	VARCHAR(50)	Utilizador dono da tarefa
titulo	VARCHAR(200)	Descrição da tarefa
data	DATE	Data prevista (opcional)
concluido	TINYINT(1)	0 = pendente, 1 = concluída

Campo	Tipo	Descrição
criado_em	DATETIME	Data/hora de criação

6. Design da Interface (UI/UX)

6.1 Estrutura Geral

A interface é composta por um cabeçalho fixo com logótipo, navegação e botão de sessão, e uma área de conteúdo principal. O design é responsivo e adaptado a ecrãs de secretária e telemóvel.

As cores principais são azul escuro (#0b1b3a) e branco, com acentos verdes (#16a34a) para presença/sucesso e vermelhos (#dc2626) para ausência/erro.

6.2 Página Principal

A página principal é o ponto de entrada do sistema. Apresenta:

- Saudação temporal ("Bom dia / Boa tarde / Boa noite")
- Carrossel de notícias da escola (geridas pelo administrador)
- Ligações para plataformas externas (SIGE, Inovar, Google Classroom)
- Botão de login no cabeçalho

6.3 Área do Aluno

Organizada em secções verticais:

- **Perfil** — nome, turma, número, badge de presença
- **Avaliações** — cartões de médias por matéria + gráfico de barras +

tabela completa de notas com badges coloridos (verde ≥ 14 , amarelo ≥ 10 , vermelho < 10)

- **Agenda** — checklist com adição, toggle de conclusão e eliminação
- **Refeições** — grelha dos próximos 5 dias úteis com toggle de marcação
- **Assiduidade** — gráfico doughnut com percentagem de presença (14 dias)
- **Histórico** — tabela de presenças com horas de entrada e saída

6.4 Área do Professor

- **Perfil** — dados profissionais
- **Presenças em tempo real** — doughnut animado + tabela com AJAX
- **Avaliações** — formulário de lançamento + tabela de notas com eliminar
- **Sumários** — formulário + lista cronológica com eliminar
- **Agenda** — checklist pessoal
- **Horário** — tabela por dia + formulário de adição/remoção de aulas
- **PDF e impressão** — exportação da lista de presenças

6.5 Área de Administração

Painel com barra lateral e nove secções:

- **Dashboard** — quatro cartões de estatísticas + gráfico de tendência 7d
- **Alunos** — CRUD com formulário inline
- **Professores** — CRUD com formulário inline
- **Cartões RFID** — associação de UID e toggle de ativo/desativado
- **Notícias** — gestão do conteúdo da página principal

- **Sumários** — consulta global com filtro por turma
- **Notas** — consulta global com filtro por turma
- **Refeições** — marcações por data + resumo semanal
- **Logs** — visualizador de logs com destaque por nível

6.6 Perfil Individual do Aluno (student_view.php)

Acessível a professores e administradores através da lista de alunos.

Apresenta toda a informação do aluno: perfil, avaliações com médias, estatísticas de assiduidade, gráfico visual de 30 dias e histórico detalhado.

Permite justificar faltas diretamente na tabela. Exportável em PDF.

6.7 Horário Semanal (dashboard.php)

Grelha de cinco colunas (Segunda a Sexta). O dia atual é destacado.

A aula que decorre no momento é assinalada com o indicador "A decorrer".

O professor e o aluno veem automaticamente o horário da sua turma.

O administrador pode seleccionar qualquer turma.

7. Diagramas

7.1 Diagrama de Casos de Uso

Aluno:

- Autenticar no sistema

- Consultar perfil
- Ver avaliações e médias
- Gerir agenda pessoal
- Marcar refeições
- Consultar horário
- Ver histórico de presenças

Professor:

- Autenticar no sistema
- Ver presenças em tempo real
- Lançar avaliações
- Escrever sumários de aulas
- Justificar faltas
- Gerir agenda pessoal
- Editar horário da turma
- Ver perfil individual de aluno
- Exportar lista em PDF

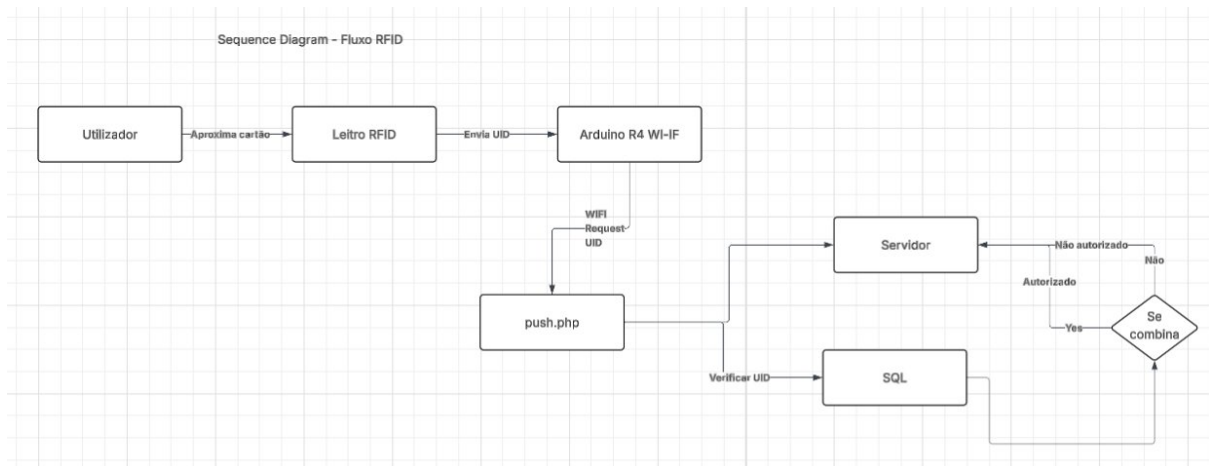
Administrador:

- Autenticar no sistema
- CRUD de alunos e professores
- Gerir cartões RFID
- Consultar notas, sumários, refeições
- Publicar e gerir notícias
- Visualizar logs do sistema

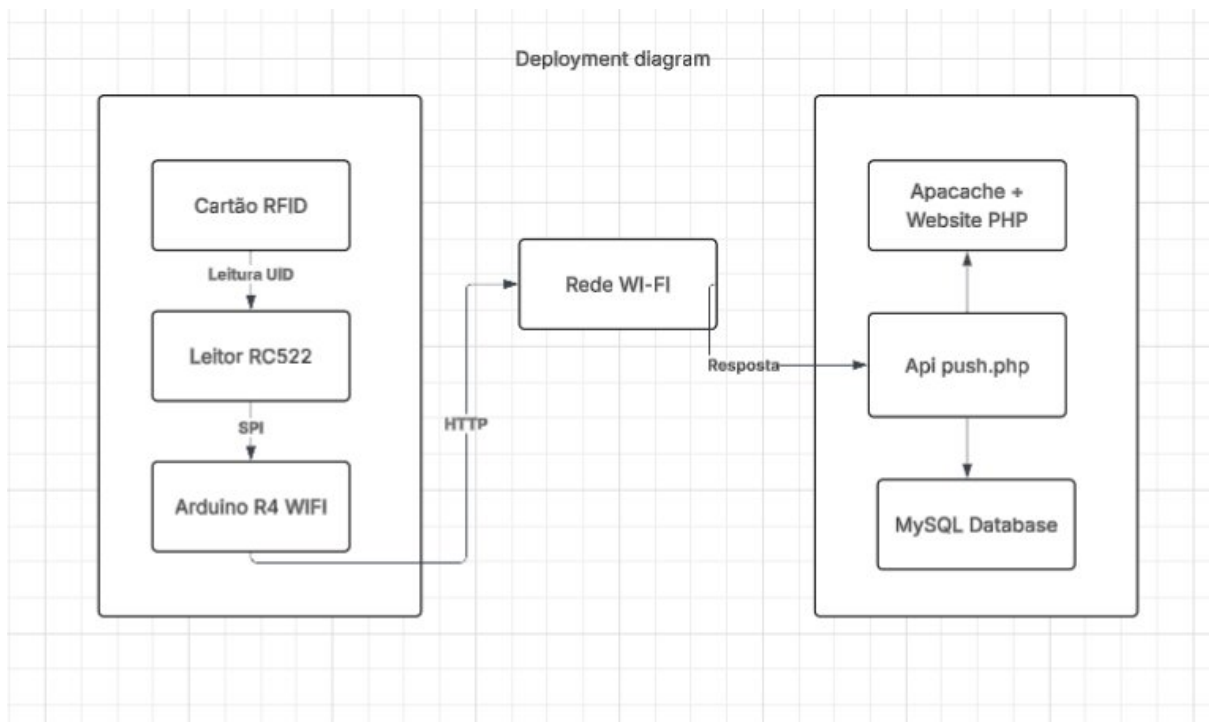
Arduino (ator externo):

- Ler UID de cartão RFID
- Enviar pedido de presença (push.php)
- Receber e interpretar resposta JSON
- Sinalizar resultado via LED RGB

7.2 Diagrama de Sequência — Fluxo de Presença RFID



7.3 Diagrama de Deployment



8. Testes

8.1 Teste do Sistema RFID

Foram realizados testes de leitura de diferentes cartões RFID para

verificar a correta captura do UID pelo módulo RC522 e o seu envio ao servidor. Os testes confirmaram que o sistema identifica corretamente os cartões e comunica com o backend sem erros.

Foram testados os seguintes cenários:

- Cartão registado e ativo → entrada registada, LED verde 1×
- Segunda leitura do mesmo cartão → saída registada, LED verde 2×
- Cartão desativado pelo administrador → HTTP 403, LED vermelho 5×
- Cartão não registado na base de dados → HTTP 404, LED vermelho 3×
- Sem ligação Wi-Fi → tentativa de reconexão automática, LED azul 2×

8.2 Teste da Comunicação Arduino-Servidor

Foi testada a comunicação HTTP entre o Arduino e o servidor PHP. Foram verificados o envio correto dos campos `uid`, `device_id` e `key`, bem como a receção e interpretação da resposta JSON pelo Arduino sem recurso a bibliotecas externas de parsing.

8.3 Teste da Base de Dados

Foram testados os seguintes cenários:

- Associação correta entre UID e utilizador
- Alternância do campo `Presença` em cada leitura
- Inserção de registos em `presencas` com data e hora corretas
- Unicidade de marcações de refeição (restrição UNIQUE na tabela)

8.4 Teste da Interface Web

Foram testadas todas as páginas do sistema para cada perfil de utilizador:

- Fluxo de autenticação: login correto, password errada, logout
- Página do aluno: visualização de notas, agenda, refeições, histórico
- Página do professor: lançamento de notas, sumários, horário
- Painel de administração: CRUD de alunos e professores, gestão de cartões
- Segurança: verificação de que um professor não consegue aceder a dados

de alunos de outras turmas

- Proteção CSRF: submissão de formulários sem token resulta em rejeição

9. Segurança

9.1 Autenticação

As passwords dos utilizadores são armazenadas exclusivamente como hash

bcrypt, gerado com `password_hash()` em PHP. A verificação no login é feita com `password_verify()`, que é resistente a timing attacks.

Nunca são guardadas passwords em texto simples na base de dados.

9.2 Proteção CSRF

Todos os formulários POST da aplicação incluem um campo oculto `_csrf` com um token aleatório de 64 caracteres hexadecimais, gerado com

`random_bytes(32)`. O servidor verifica o token com `hash_equals()` antes de processar qualquer pedido POST. Pedidos sem token válido são rejeitados com HTTP 403.

9.3 Prevenção de SQL Injection

Toda a aplicação usa exclusivamente prepared statements PDO com parâmetros vinculados. Nunca é concatenado input do utilizador diretamente em queries SQL. A opção `PDO::ATTR_EMULATE_PREPARES = false` garante que as preparações são enviadas ao servidor MySQL como operações reais.

9.4 Prevenção de XSS

Todos os valores apresentados ao utilizador no HTML passam pela função `htmlspecialchars()`. O helper `h()` no painel de administração torna este processo consistente em todo o código.

9.5 Controlo de Acesso

Cada página verifica o role do utilizador em sessão logo no início. O professor está restrito aos alunos da sua própria turma — esta verificação é feita tanto no GET (ao carregar a página) como no POST (ao submeter formulários). O administrador tem acesso total.

9.6 Segurança das Sessões

As sessões PHP são configuradas com:

- `httponly = true` — o cookie de sessão não é acessível a JavaScript
- `samesite = Strict` — previne envio do cookie em pedidos cross-site
- `session_destroy()` no logout — elimina todos os dados de sessão

9.7 Proteção da API RFID

O endpoint da API está protegido por uma API key partilhada entre o Arduino

e o servidor. Pedidos sem a chave correta são rejeitados com HTTP 401.

Os campos sensíveis (como a API key) são automaticamente mascarados nos

logs pelo módulo `logger.php`.

9.8 Estrutura dos Logs

O sistema regista eventos em ficheiros JSON diários (`logs/app-YYYY-MM-DD.log`),

localizados fora do diretório `public/` e portanto inacessíveis via browser.

Cada entrada inclui:

- `time` — data e hora do evento
- `level` — INFO / WARN / ERROR
- `ip` — endereço IP do pedido
- `uri` — endpoint chamado
- `msg` — mensagem descritiva
- `ctx` — contexto adicional (uid, login, resultado, etc.)

10. Implementação

10.1 Endpoint de Presença (`api/push.php`)

O endpoint executa a seguinte lógica:

1. Recebe `uid`, `key` e `device_id` via HTTP POST
2. Verifica a API key — rejeita com HTTP 401 se inválida
3. Normaliza o UID (maiúsculas, sem espaços)
4. Procura o UID na tabela login
5. Verifica se o cartão está ativo — rejeita com HTTP 403 se não estiver
6. Conforme o Role, acede a alunos ou professores
7. Inverte o campo Presença (toggle)
8. Insere registo em presencas com data, hora e `device_id`
9. Devolve JSON com `{ok: true, estado: "entrada"/"saida", nome, turma}`

10.2 Endpoint AJAX de Presenças (`api/presence.php`)

Endpoint GET que recebe o parâmetro `turma` e devolve o estado atual de presença de todos os alunos dessa turma. Usado pelo frontend para atualização automática a cada 5 segundos sem recarregar a página. Só acessível a utilizadores com role `Professor` ou `admin`.

10.3 Padrão PRG (Post-Redirect-Get)

Toda a aplicação web usa o padrão PRG: os formulários POST processam a ação e fazem imediatamente `header('Location: ...')`. Isto impede que o utilizador resubmeta acidentalmente um formulário ao recarregar a página.

10.4 Firmware Arduino

O firmware lê o UID do cartão RFID e envia-o ao servidor. Distingue os seguintes resultados através do LED RGB integrado:

Situação	Sinal LED
Entrada registada (HTTP 200)	1 flash verde longo (600ms)
Saída registada (HTTP 200)	2 flashes verdes curtos
Cartão desativado (HTTP 403)	5 flashes vermelhos rápidos
UID não encontrado (HTTP 404)	3 flashes vermelhos
Erro de API key (HTTP 401)	3 flashes vermelhos
Sem ligação Wi-Fi	2 flashes azuis

O dispositivo usa o MAC address como `device_id`, permitindo identificar qual terminal registou cada presença. O anti-spam impede que o mesmo cartão seja processado duas vezes em menos de 10 segundos.

11. Deploy e Ambiente

O sistema foi desenvolvido e testado em ambiente local utilizando WAMP

(Windows + Apache + MySQL + PHP), que inclui servidor Apache, base de dados MySQL 9.1 e PHP 8.3.

Para instalar o sistema:

1. Copiar a pasta do projeto para `C:\wamp64\www\PAP\`
2. Executar `db/schema_full.sql` no phpMyAdmin para criar as 9 tabelas
3. Executar `db/seed_data.sql` para carregar dados de teste
4. Aceder em `http://localhost/PAP/project/public/index.php`

Para o Arduino:

1. Abrir `arduino/PAP_RFID/PAP_RFID.ino` no Arduino IDE 2.x
2. Configurar SSID, password Wi-Fi e IP do servidor
3. Selecionar placa Arduino UNO R4 WiFi
4. Instalar bibliotecas: MFRC522, WiFiS3, ArduinoHttpClient
5. Carregar o sketch

Em contexto de produção, o sistema pode ser migrado para um servidor

Apache/Nginx com PHP 8.x e MySQL 8+ sem alterações no código. A

configuração de ligação à base de dados está centralizada em `config/db.php`.

12. Logging e Monitorização

O sistema possui logging em duas camadas independentes.

12.1 Logging no Arduino

O Arduino apresenta informação em tempo real no Serial Monitor:

- Detecção de cartão e UID lido
- Estado da ligação Wi-Fi
- Código HTTP e resposta do servidor
- Resultado interpretado ("ENTRADA registada", "SAIDA registada", etc.)

12.2 Logging no Backend PHP

O módulo `api/lib/logger.php` escreve entradas JSON em ficheiros diários em `logs/app-YYYY-MM-DD.log`. Os ficheiros de log ficam fora do diretório público, não sendo acessíveis via browser.

São registados: pedidos RFID recebidos, correspondências de utilizador, tentativas de autenticação, erros e avisos. Campos sensíveis como a API key e a password são automaticamente mascarados.

O administrador pode consultar os logs diretamente no painel de administração, com destaque visual por nível de severidade.

13. Manutenção e Evolução

O sistema foi desenvolvido com uma arquitectura modular que facilita a manutenção e a adição de novas funcionalidades. O código está organizado de forma clara, com separação entre configuração, lógica de negócio e apresentação.

Possíveis melhorias futuras:

- **Notificações:** envio de alertas aos encarregados de educação por e-mail ou SMS em caso de ausência não justificada
- **Aplicação móvel:** interface nativa para iOS e Android
- **Relatórios em PDF:** exportação automática de relatórios mensais de assiduidade por turma
- **Integração com plataformas externas:** sincronização automática com SIGE ou Inovar
- **Múltiplos terminais:** suporte a vários dispositivos Arduino em diferentes entradas do edifício
- **Painel de encarregado de educação:** acesso restrito para consulta da presença e avaliações do educando

14. Conclusão

O projeto desenvolvido resultou num sistema funcional completo que integra um website escolar com tecnologia RFID. A solução cobre o ciclo completo de gestão de uma turma: controlo de presenças automático, lançamento de

avaliações, diário de aulas, gestão de horários e agenda pessoal.

A integração entre hardware e software foi concretizada com sucesso: o dispositivo Arduino R4 WiFi lê cartões RFID e comunica com o servidor PHP em tempo real, distinguindo visualmente diferentes situações através de padrões de LED.

Do ponto de vista técnico, o projeto aplica boas práticas de segurança (bcrypt, CSRF, prepared statements, controlo de acesso por role) e padrões de desenvolvimento web (PRG, separação de responsabilidades, logging estruturado).

O sistema está pronto para uso em ambiente escolar real e apresenta uma base sólida para evolução futura.